

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

(повне найменування назва факультету)

Кафедра дискретного аналізу та інтелектуальних систем

(повна назва кафедри)

Курсова робота

РОЗРОБКА СИСТЕМИ ДЛЯ АВТОМАТИЧНОГО ВИЯВЛЕННЯ ФЕЙКОВИХ НОВИН

Виконав: студент групи ПМі-33с
спеціальності

122 «Комп'ютерні науки»

(шифр і назва спеціальності)

Яценко В.Т.

(підпис)

(прізвище та ініціали)

Керівник

Пелюшкевич О. В.

(підпис)

(прізвище та ініціали)

Рецензент

(підпис)

(прізвище та ініціали)

ЗМІСТ

ЗМІСТ.....	2
ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ І ТЕРМІНІВ	3
ВСТУП	5
1 ПОСТАНОВКА ЗАДАЧІ ТА ОГЛЯД НАЯВНИХ РІШЕНЬ.....	8
1.1 Постановка задачі.....	8
1.2 Огляд наявних рішень.....	9
1.3 Проєктування вимог.....	11
2 ВИКОРИСТАНІ ТЕХНОЛОГІЇ ТА АЛГОРИТМИ.....	16
2.1 Математичні основи попередньої обробки та векторизації тексту.....	16
2.2 Базові моделі машинного навчання.....	18
2.3 Архітектури глибокого навчання на основі трансформерів	20
2.4 Формування датасету	26
2.5 Програмний інструментарій та засоби розробки.....	30
3 РЕАЛІЗАЦІЯ СИСТЕМИ ТА РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ	32
3.1 Базова модель детекції фейкових новин (Baseline)	32
3.2 Детекція фейкових новин на основі mBERT	36
3.3 Експеримент А: Вплив дисбалансу класів на якість детекції фейків 41	
3.4 Базова модель тематичної класифікації (LinearSVC)	44
3.5 Тематична класифікація на основі mBERT	48
3.6 Реалізація вебдодатку.....	57
СПИСКИ ВИКОРИСТАНИХ ДЖЕРЕЛ	58

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ І ТЕРМІНІВ

TF-IDF (англ. TF - term frequency, IDF - inverse document frequency) - статистична міра для оцінки важливості слова в документі, що є частиною колекції або корпусу.

Векторизація - процес перетворення текстових даних у числовий формат

Accuracy - точність класифікації, частка правильно класифікованих зразків

Precision - точність, частка правильно визначених позитивних прикладів серед усіх визначених як позитивні

Recall - повнота, частка правильно визначених позитивних прикладів серед усіх фактично позитивних

F1-score - гармонійне середнє між precision та recall

Baseline (базова лінія) - початкова модель з простою архітектурою що використовується як точка відліку для порівняння з більш складними рішеннями

Лематизація - процес приведення слів до їх базової форми (леми)

Токенізація - процес розбиття тексту на окремі одиниці (токени), зазвичай слова або символи

BERT (англ. Bidirectional Encoder Representations from Transformers) - двонаправлена трансформерна модель для отримання контекстуальних векторних представлень тексту.

mBERT (англ. Multilingual BERT) - багатомовна версія моделі BERT.

Трансформер (англ. Transformer) - архітектура нейронної мережі на основі механізму самоуваги, що обробляє всі токени послідовності паралельно без використання рекурентності.

Self-Attention (механізм самоуваги) - механізм, за якого кожен токен послідовності зважено враховує всі інші токени при формуванні свого векторного представлення.

NLP (англ. Natural Language Processing) - обробка природної мови; галузь штучного інтелекту, що займається автоматичним аналізом та генерацією текстів.

MLM (англ. Masked Language Modeling) - задача передтренування BERT: модель передбачає випадково замасковані токени на основі двобічного контексту.

NSP (англ. Next Sentence Prediction) - друга задача передтренування

SVM (англ. Support Vector Machine) - метод опорних векторів; алгоритм машинного навчання для класифікації, що знаходить оптимальну гіперплощину розділення класів.

Embedding (векторне представлення) - відображення слів або токенів у щільний числовий вектор у просторі семантичних ознак.

ReLU (Rectified Linear Unit) - функція, яка пропускає додатні числа без змін, а всі від'ємні перетворює на нуль.

Softmax - функція, яка перетворює набір чисел на ймовірності (від 0 до 1), сума яких завжди дорівнює одиниці (100%).

GELU (Gaussian Error Linear Unit) - більш плавна (згладжена) версія ReLU. Є базовим стандартом для трансформерів, оскільки краще працює зі складними даними.

Data-Centric AI - підхід до розробки штучного інтелекту, де головна увага приділяється покращенню якості та структури даних, а не ускладненню архітектури моделі.

ВСТУП

Сучасний інформаційний простір характеризується безпрецедентними обсягами та швидкістю поширення даних. Соціальні мережі, онлайн-медіа та месенджери стали основними джерелами новин для значної частини населення, що створює як нові можливості для доступу до знань, так і суттєві виклики у контексті оцінки якості та достовірності контенту. По-перше, традиційні механізми перевірки інформації не встигають за темпами її поширення в цифровому середовищі. По-друге, поряд із достовірною інформацією цими ж каналами активно поширюється дезінформація, маніпулятивний контент та фейкові новини. Це явище становить серйозну загрозу для суспільної стабільності, демократичних процесів та національної безпеки, особливо в умовах гібридних війн та цілеспрямованих інформаційних атак. За даними дослідження Reuters Institute Digital News Report 2023[1], яке охоплює 46 країн світу, 56% користувачів стурбовані тим, що не можуть відрізнити правдиву інформацію від фейкової в інтернеті. В Україні ситуація є особливо критичною через триваючу інформаційну війну, що супроводжує військову агресію, де дезінформація використовується як інструмент гібридної війни для підризу довіри до державних інституцій та міжнародної підтримки. Водночас, зростаюча роль генеративних моделей штучного інтелекту у створенні правдоподібних, але фактично хибних повідомлень ускладнює ситуацію, роблячи фейкові новини все складнішими для розпізнавання людиною.

Сучасний стан проблеми розкривається в низці викликів, таких як обмежена масштабованість традиційного фактчекінгу, який виконується експертами вручну, відсутність єдиних стандартів перевірки достовірності інформації в україномовному сегменті, та необхідність швидкої реакції на інформаційні атаки. Традиційні методи верифікації новин часто не встигають за темпами генерації та поширення неправдивого контенту. Ці проблеми підкреслюють необхідність розвитку нових, автоматизованих підходів до виявлення фейкових новин, які б забезпечували швидкість, точність та масштабованість аналізу.

Актуальність даної роботи полягає у необхідності розробки та впровадження нових технологічних рішень, спрямованих на автоматизацію процесів виявлення недостовірної інформації. Технології машинного навчання та обробки природної мови можуть стати відповіддю на вищезгадані проблеми, гарантуючи швидку та ефективну фільтрацію інформаційних потоків. Такі системи дозволяють створювати моделі, що розрізняють характеристики правдивих та фейкових повідомлень на основі великих обсягів даних, сприяючи підвищенню стійкості суспільства до дезінформації.

Мета даної роботи полягає у розробці та реалізації моделей машинного навчання для автоматичного виявлення фейкових новин у текстових даних. Ця система має на меті забезпечити точну класифікацію новин за достовірністю та тематикою, одночасно акцентуючи увагу на доступності та зручності у використанні для кінцевих користувачів через вебінтерфейс.

Розробка системи виявлення фейкових новин передбачає комплексний аналіз існуючих підходів та методів, виявлення прогалин та можливостей для вдосконалення. Також вона включає оцінку різних технологій машинного навчання, зокрема методів векторизації тексту, алгоритмів класифікації та архітектур нейронних мереж, на основі точності, швидкості роботи та придатності для україномовних даних. Вибір цих технологій базується на вимогах і цілях проєкту. Створення системи включає етапи збору та попередньої обробки даних, побудови та тренування моделей, тестування та розгортання. Користувацький інтерфейс також детально описується, включаючи функціональні можливості та принципи взаємодії з користувачем, щоб забезпечити інтуїтивно зрозумілу та доступну систему для всіх.

Основними завданнями даної курсової роботи є дослідження існуючих підходів до автоматичного виявлення фейкових новин та тематичної класифікації тексту, збір та підготовка корпусу україномовних новин, статистичний аналіз текстових даних, розробка базової моделі на основі логістичної регресії з використанням TF-IDF векторизації, дослідження більш складних архітектур, зокрема нейронних мереж, оцінка ефективності розроблених моделей за

допомогою метрик якості (accuracy, precision, recall, F1-score), та створення вебдодатку для демонстрації роботи моделей.

Результатом роботи є вебдодаток для виявлення фейкових новин, який є зручною платформою, здатною аналізувати текстовий контент та надавати прогноз щодо його достовірності. Систему можна застосовувати в медіа-моніторингу, на соціальних платформах, в освітніх закладах та для особистого використання. Медіа-компанії можуть використовувати її для попередньої фільтрації контенту та маркування потенційно недостовірної інформації, а освітні заклади - для підвищення медіаграмотності студентів. Дослідники медіа можуть застосовувати систему для аналізу інформаційних потоків та вивчення механізмів поширення дезінформації. Це ключовий момент, який допомагає суспільству протистояти інформаційним маніпуляціям та розвивати критичне мислення при споживанні новин.

1 ПОСТАНОВКА ЗАДАЧІ ТА ОГЛЯД НАЯВНИХ РІШЕНЬ

1.1 Постановка задачі

У курсовій роботі було поставлено за мету розробити систему автоматичного виявлення фейкових новин для україномовного контенту з використанням методів машинного навчання та обробки природної мови, з огляду на складну морфологію та лексико-семантичні особливості української мови.

Система має забезпечити бінарну класифікацію новин (правдива/фейкова) та додаткову тематичну категоризацію контенту, надаючи користувачам швидкий та об'єктивний інструмент для оцінки достовірності інформації через зручний вебінтерфейс. Для досягнення поставленої мети необхідно вирішити наступні задачі:

- Збір та формування якісного корпусу україномовних новин з верифікованими мітками достовірності (правдива/фейкова) та тематичними категоріями, що є критично важливим етапом через загальний дефіцит відкритих та структурованих україномовних датасетів.
- Дослідження та імплементація методів попередньої обробки текстових даних (токенізація, видалення стоп-слів, лематизація та векторизація) з обов'язковою адаптацією алгоритмів до багатой словозміни, складного словотворення та вільного синтаксису, притаманних українській мові.
- Розробка базової моделі класифікації на основі логістичної регресії з використанням TF-IDF векторизації для встановлення baseline метрик якості та виявлення маркерної лексики.
- Дослідження та імплементація сучасних архітектур машинного навчання на базі трансформерів (зокрема, багатомовних моделей класу mBERT), які здатні розуміти глибинний контекст тексту та виявляти специфічні маніпулятивні патерни україномовного інфопростору, нівелюючи проблему багатозначності слів.
- Проведення порівняльного аналізу розроблених моделей за метриками accuracy, precision, recall та F1-score на тестовій вибірці.

- Розробка вебдодатка з інтуїтивним користувацьким інтерфейсом для демонстрації роботи моделей, що дозволяє користувачам вводити текст новини та отримувати прогноз щодо її достовірності та тематики.

1.2 Огляд наявних рішень

Сучасні підходи до верифікації інформації та виявлення дезінформації можна класифікувати за кількома критеріями: рівнем автоматизації, цільовою аудиторією та використаними технологіями. Глобально їх можна розділити на традиційний (експертний) фактчекінг, інтерактивні інструменти, краудсорсингові платформи та системи на базі машинного навчання.

Професійні фактчекінгові платформи та агрегатори

Традиційний фактчекінг базується на методології відкритої розвідки (OSINT) та ручному аналізі джерел журналістами-дослідниками. На глобальному рівні стандартом агрегації таких перевірок є Fact Check Explorer [2] від компанії Google. Це пошукова система, яка збирає результати від понад 100 міжнародних організацій, використовуючи спеціалізовану мікророзмітку ClaimReview. Інструмент дозволяє шукати спростування за ключовими словами, особами або URL-адресами. Для інтеграції у сторонні сервіси Google пропонує Fact Check Tools API [3]. Міжнародні платформи, такі як американський PolitiFact [8], використовують систему наочних рейтингів (від «True» до «Pants on Fire») для оцінки достовірності політичних заяв.

Головним недоліком цього підходу є часовий лаг: ручна перевірка потребує годин або навіть днів, тоді як фейкова новина поширюється мережею за лічені хвилини.

Український сегмент ручного фактчекінгу

В умовах інтенсивної інформаційної війни в Україні провідну роль відіграють спеціалізовані організації. Проєкт StopFake [4], заснований у 2014 році, фокусується на деконструкції російської пропаганди та спростуванні дезінформації у ЗМІ. Проєкт VoxCheck [5] (від «Детектор медіа») спеціалізується на фактчекінгу заяв українських політиків та економічних маніпуляцій. Ініціатива «Перевірка» від

Gwara Media [7] функціонує як Telegram-бот, куди користувачі надсилають підозрілі посилання, після чого команда аналітиків обробляє їх у напіваавтоматичному режимі (Human-in-the-loop). Попри високу точність та експертність, усі ці проєкти стикаються з проблемою масштабованості через обмеженість людського ресурсу.

Інтерактивні та краудсорсингові підходи

Для підвищення медіаграмотності населення створюються інтерактивні рішення. Український сервіс FakeCheck.in.ua [6] пропонує користувачам пройти чек-лист (оцінити джерело, емоційність заголовка, наявність автора), після чого алгоритм видає ймовірнісну оцінку достовірності тексту. Однак цей метод є евристичним і покладається на суб'єктивне сприйняття читача, а не на реальний аналіз даних.

Автоматизовані системи та машинне навчання (NLP)

Останньою категорією є спроби повної автоматизації верифікації за допомогою методів обробки природної мови (NLP). Дослідницький проєкт Університету Саймона Фрейзера [9] та подібні академічні ініціативи демонструють ефективність використання класичного машинного навчання (наприклад, алгоритмів SVM або випадкового лісу з TF-IDF векторизацією) для аналізу англomовних новин.

Висновки щодо існуючих рішень

Аналіз наявного програмного забезпечення та досліджень виявляє критичну прогалину в сфері протидії дезінформації:

1. Традиційні сервіси не здатні масштабуватися для аналізу великих потоків даних (наприклад, у Telegram-каналах) у режимі реального часу.
2. Інтерактивні інструменти залежать від компетенції самого користувача.
3. Існуючі автоматизовані ML-рішення демонструють різке падіння ефективності (Accuracy) при роботі з україномовним контентом. Це зумовлено складною морфологією української мови та специфікою локальних маніпулятивних патернів, які неможливо виявити за допомогою базового словникового аналізу (Bag-of-Words).

Вирішення цієї проблеми вимагає переходу від класичних статистичних моделей до використання глибоких нейронних мереж та архітектур на базі трансформерів (зокрема, багатомовних моделей класу BERT), здатних розуміти глибинний контекст українського тексту.

1.3 Проектування вимог

Вимоги до програмного забезпечення визначають характеристики та обмеження системи, які необхідно реалізувати для досягнення поставленої мети. Вони поділяються на функціональні, що описують конкретні можливості та поведінку системи, та нефункціональні, що визначають якісні характеристики та обмеження, за яких система має функціонувати. Чітко сформульовані вимоги є основою для проектування архітектури, розробки, тестування та оцінки успішності проєкту.

Для візуалізації функціональних вимог системи розроблено UML діаграму прецедентів, яка представлена на рисунку 1.1.

Діаграма прецедентів відображає основні сценарії взаємодії користувача з системою виявлення фейкових новин. Архітектура системи логічно розділена на дві взаємопов'язані підсистеми: введення та обробка (ліва частина діаграми) та аналіз результатів (права частина діаграми).

Користувач виступає єдиним актором системи та має доступ до всього функціоналу через вебінтерфейс. Взаємодія користувача з системою відбувається через десять основних прецедентів, які охоплюють повний цикл роботи з новинним контентом - від введення даних до отримання результатів аналізу.

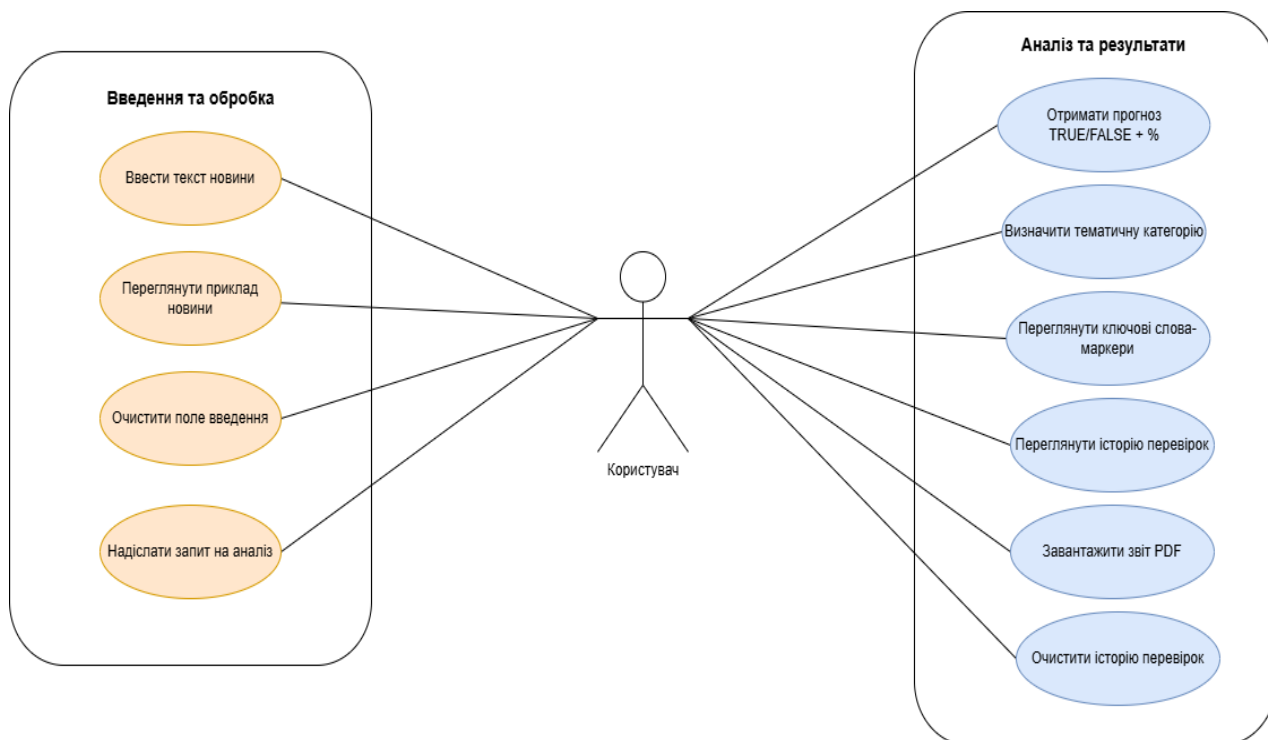


Рисунок 1.1 – Діаграма прецедентів

Функціональні вимоги визначають конкретні функції, операції та можливості, які система має надавати користувачам. Вони описують, що саме система повинна робити у відповідь на певні входні дані або в певних ситуаціях.

Система має забезпечувати наступні функціональні можливості:

- Бінарна класифікація новин за критерієм достовірності (правдива/фейкова) з точністю не менше 80% на тестовій вибірці.
- Тематична категоризація контенту для забезпечення контекстуального аналізу та можливості фільтрації результатів за предметними областями.
- Обробка україномовного тексту з підтримкою різних стилів написання (формальний/неформальний, новинний/блоговий формат).
- Попередня обробка вхідного тексту, що включає: токенизацію українського тексту, нормалізацію (приведення до нижнього регістру), видалення стоп-слів на основі українського словника, лематизацію, та векторизацію.
- Вебінтерфейс має надавати можливість введення тексту новини через текстове поле з підтримкою копіювання/вставки контенту.

- Відображення результатів аналізу для кожної новини у форматі карточки або таблиці з чіткою індикацією: відсоток ймовірності фейковості (0-100%), визначена тематична категорія.
- Збереження історії перевірених новин у сесії користувача для можливості порівняння результатів.
- Експорт результатів аналізу у форматі CSV або Excel з включенням наступних полів: оригінальний текст новини, прогноз достовірності (FAKE/TRUE), ймовірність фейковості (%), тематична категорія, дата та час перевірки.
- Адаптивний інтерфейс, що коректно відображається на різних пристроях (десктоп, планшет, смартфон).

Нефункціональні вимоги визначають якісні характеристики системи, такі як продуктивність, надійність, безпека, масштабованість та зручність використання. Вони описують не що система робить, а як вона це робить, встановлюючи стандарти якості та обмеження, за яких система має функціонувати.

Система має відповідати наступним нефункціональним вимогам:

1. Продуктивність та масштабованість:

- Час відгуку вебінтерфейсу - не більше 1 секунди при стандартному інтернет-з'єднанні.
- Архітектура має забезпечувати можливість горизонтального масштабування для обробки зростаючих обсягів даних.

2. Надійність та доступність:

- У разі виникнення помилки обробки (некоректний формат даних, занадто короткий текст, технічний збій) система має відображати зрозуміле повідомлення для користувача без розкриття внутрішніх технічних деталей або трасування стеку помилок.
- Передбачити обробку виключних ситуацій (некоректний формат вхідних даних, обрив з'єднання) з коректним інформуванням користувача.

- Система має зберігати працездатність при роботі з текстами, що містять помилки правопису, опечатки, повтори символів або нестандартні розділові знаки без критичних збоїв.

3. Зручність використання:

- Інтерфейс має бути інтуїтивно зрозумілим для користувачів без технічної підготовки або попереднього досвіду роботи з системами машинного навчання.
- Інтерактивні елементи мають мати зрозумілі підказки та описи.
- Швидкий час навчання для роботи з системою новому користувачу.
- Інтерфейс має бути українською мовою з чіткими формулюваннями всіх елементів без технічного жаргону.

4. Підтримка та розширюваність:

- Вихідний код має бути опублікований як open-source на платформі GitHub для можливості подальшого розвитку дослідниками та розробниками.
- Код має бути задокументований з коментарями для ключових функцій та класів згідно зі стандартом PEP 257 для Python docstrings.
- README файл у репозиторії має містити: детальні інструкції з встановлення залежностей, крок-за-кроком гайд з розгортання системи локально та на сервері, опис структури проєкту та призначення кожного модуля.

5. Сумісність:

- Вебдодаток має коректно працювати у сучасних браузерях останніх двох версій.
- Підтримка роботи на операційних системах Windows, macOS, Linux.

6. Етичні вимоги та прозорість:

- Рішення моделей машинного навчання мають супроводжуватися поясненнями (Explainable AI) через відображення найбільш значущих слів та фраз, що вплинули на класифікацію.
- Система має включати чітко видиме застереження про те, що результати є автоматичними прогнозами моделей машинного навчання і можуть містити помилки, рекомендуючи додаткову перевірку інформації через офіційні джерела для критично важливих ситуацій.
- Результати класифікації не мають представлятися як абсолютна істина, а лише як допоміжний інструмент для користувача у процесі критичного осмислення інформації.

Сформульовані вимоги слугують основою для проєктування архітектури системи, вибору технологій та методів машинного навчання, а також для подальшого тестування та валідації розробленого рішення. Дотримання цих вимог забезпечить створення функціональної, надійної та зручної системи для автоматичного виявлення фейкових новин, яка відповідатиме потребам українських користувачів у боротьбі з дезінформацією в сучасному цифровому інформаційному просторі.

2 ВИКОРИСТАНІ ТЕХНОЛОГІЇ ТА АЛГОРИТМИ

2.1 Математичні основи попередньої обробки та векторизації тексту

Машинні алгоритми не здатні безпосередньо аналізувати символічні рядки, тому процес виявлення маніпуляцій у новинах починається з трансформації тексту у математичні структури. Цей етап включає кілька послідовних кроків: токенизацію, очищення від стоп-слів та лематизацію.

Токенизація відповідає за розбиття суцільного тексту на окремі слова. Після цього з тексту вилучаються стоп-слова - елементи мови, які не несуть важливого семантичного навантаження. Для зменшення розмірності простору ознак застосовується лематизація, тобто зведення всіх слів до їхньої словникової форми (леми).

Після морфологічної обробки застосовується векторизація тексту. У даній роботі використано міру TF-IDF. Цей підхід складається з двох компонентів:

1. Термінна частота (TF) - частота терміну. Визначає, наскільки часто слово t зустрічається в конкретному документі d . Чим частіше слово зустрічається в новині, тим воно важливіше для цього конкретного тексту:

$$TF(t, d) = \frac{n_t}{\sum_k n_k}$$

2. Обернена частота документа (IDF, Inverse Document Frequency) — зменшує вагу слів, що зустрічаються майже в усіх новинах, і збільшує вагу специфічних слів, характерних для фейків:

$$IDF(t, D) = \log\left(\frac{|D|}{|\{d \in D : t \in d\}|}\right)$$

Результуюча вага слова у документі є добутком цих двох величин: $W_{t,d} = TF \times IDF$. Це дозволяє виділити специфічні слова-маркери, які найчастіше зустрічаються саме у фейкових або, навпаки, у правдивих новинах.

На відміну від класичних алгоритмів, що базуються на мірі TF-IDF та потребують жорсткої лематизації тексту, архітектури на базі трансформерів (зокрема mBERT) використовують власні механізми попередньої обробки. Для

підготовки даних застосовується алгоритм токенизації підслів, такий як WordPiece. Цей алгоритм розбиває невідомі або складні слова на відомі частини (наприклад, корінь та суфікс), що дозволяє моделі ефективно працювати з багатою морфологією української мови без втрати семантики та без створення словників нескінченного розміру. Додатково, на етапі токенизації для кожної новини генеруються спеціальні маркери: токен початку послідовності ([CLS]), який акумулює загальний контекст для подальшої класифікації, токен розділювача ([SEP]), а також маски уваги (Attention Masks), які вказують нейромережі на ігнорування доданих нульових токенів (padding) для вирівнювання довжини вхідних векторів.

Математична модель алгоритму WordPiece

Математична суть алгоритму полягає у максимізації функції вірогідності мовної моделі на навчальному корпусі текстів. На етапі побудови словника алгоритм ітеративно об'єднує ті пари символів або підслів, які дають найбільший приріст вірогідності.

Критерій об'єднання двох сусідніх підслів x та y у новий єдиний токен xy обчислюється за формулою оцінки (Score), що базується на точковий взаємній інформації:

$$Score(x, y) = \frac{P(x, y)}{P(x)P(y)}$$

Де $P(x, y)$ - ймовірність спільної появи послідовності підслів x та y у навчальному корпусі текстів; $P(x)$ та $P(y)$ - незалежні ймовірності появи кожного окремого підслова.

Якщо значення $Score$ є максимальним серед усіх можливих пар, алгоритм зливає x та y в один токен. Завдяки цій математичній властивості, WordPiece об'єднує частини слів лише тоді, коли вони дійсно утворюють стійку морфологічну або семантичну одиницю (наприклад, корінь слова), залишаючи рідкісні слова розбитими на базові складові.

2.2 Базові моделі машинного навчання

Для вирішення поставлених завдань архітектура розроблюваної системи була розділена на два незалежні класифікаційні модулі. Замість використання єдиного універсального алгоритму, для кожної підзадачі було обрано найбільш оптимальний математичний апарат, що дозволило максимізувати загальну ефективність системи. Як базова лінія (Baseline) для порівняння з нейромережевими підходами були застосовані класичні лінійні алгоритми.

Модуль детектування достовірності

Першим алгоритмом обрано логістичну регресію, яка є стандартом для завдань бінарної класифікації [10]. У контексті даної системи вона виконує функцію модуля детектування достовірності та відповідає за визначення ймовірності того, чи є подана новина фейковою (клас 0), чи правдивою (клас 1).

Алгоритм обчислює зважену суму вхідних ознак (значень TF-IDF) та пропускає результат через логістичну функцію (сигмоїду), що має вигляд:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

де z - лінійна комбінація ознак тексту та їхніх ваг. Сигмоїда нелінійно стискає будь-яке дійсне значення z у діапазон від 0 до 1, що дозволяє інтерпретувати результат не просто як жорстку відповідь, а як статистичну ймовірність того, що новина є маніпулятивною. Якщо розрахована ймовірність перевищує встановлений поріг (за замовчуванням 0.5), система маркує контент як фейк. Основною перевагою логістичної регресії в контексті текстового аналізу є її висока інтерпретованість: аналіз вагових коефіцієнтів моделі дозволяє чітко встановити, які саме лексичні маркери алгоритм вважає найбільш вагомими для кожного з класів.

Модуль тематичної категоризації

Другим алгоритмом було обрано метод лінійної класифікації на основі опорних векторів (Linear SVC). У розробленій системі він відповідає за модуль тематичної категоризації - віднесення новини до однієї з багатьох тем (політика, спорт, технології тощо).

Support Vector Machine (SVM) [11] - це алгоритм просторового розділення даних. Завданням методу Linear SVC (з лінійним ядром) є знаходження оптимальної гіперплощини у багатовимірному просторі ознак, яка найкраще розділяє класи. Гіперплощина будується таким чином, щоб максимізувати зазор між найближчими точками різних класів. Ці найближчі точки і називаються «опорними векторами».

Для вирішення проблеми багатокласової класифікації алгоритм застосовує стратегію «один проти всіх» (One-vs-Rest). Суть підходу полягає у побудові окремого бінарного класифікатора для кожної теми, який відділяє її від усіх інших категорій. Водночас для компенсації можливого дисбалансу кількості новин у різних категоріях навчального набору даних під час ініціалізації моделі застосовується автоматичне балансування ваг класів, що запобігає зміщенню результатів моделі у бік найпопулярніших тематик.

Математично завдання Linear SVC полягає у знаходженні такої гіперплощини в багатовимірному просторі ознак (де кожна ознака - це вага слова з TF-IDF), яка розділяє класи з максимальним зазором (маржею). Рівняння розділювальної гіперплощини має вигляд:

$$\omega^T x + b = 0$$

де ω - вектор ваг моделі (вектор нормалі до гіперплощини), x - вектор вхідних ознак тексту b - скалярне зміщення (bias).

Оскільки реальні текстові дані рідко є ідеально лінійно роздільними, на практиці використовується алгоритм з м'яким зазором (Soft-Margin SVM). Процес навчання моделі зводиться до розв'язання такої оптимізаційної задачі:

$$\min_{\omega, b, \xi} \left(\frac{1}{2} \|\omega\|^2 + C \sum_{i=1}^n \xi_i \right)$$

за умов виконання обмежень:

$$y_i(\omega^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

У цих формулах:

- $\frac{1}{2} \|\omega\|^2$ - регуляризаційний член, мінімізація якого призводить до максимізації ширини зазору між класами;
- C - гіперпараметр регуляризації (штраф за помилку), який контролює баланс між шириною зазору та кількістю допущених помилок на навчальній вибірці;
- ξ_i (slack variables) - змінні послаблення, що дозволяють деяким текстам знаходитись по неправильний бік маржі (врахування шуму в даних);
- $y_i \in \{-1, 1\}$ - математичні мітки класів (наприклад, конкретна тема проти всіх інших тем).

Після того, як алгоритм знаходить оптимальні параметри ω та b , класифікація нового невідомого тексту x здійснюється шляхом обчислення знака вирішальної функції:

$$f(x) = \text{sign}(\omega^T x + b)$$

Якщо результат позитивний, новина відноситься до цільової категорії, якщо негативний - до інших.

2.3 Архітектури глибокого навчання на основі трансформерів

Класичні методи машинного навчання - логістична регресія та метод опорних векторів - ефективно обробляють текст у рамках парадигми «мішок слів» (bag-of-words). Парадигма «мішок слів» - це підхід до представлення тексту, за якого документ розглядається як неупорядкований набір слів: порядок слів, граматичні зв'язки та синтаксична структура повністю ігноруються. Наприклад, речення "Зеленський заявив що Росія бреше" та "Росія заявила що Зеленський бреше" матимуть однакове векторне представлення попри протилежний зміст. Саме це є фундаментальним обмеженням при детекції фейків, де маніпуляція реалізується через контекст, а не словниковий склад. Для подолання цього обмеження сучасна обробка природної мови спирається на архітектуру трансформера та моделі попереднього навчання на великих текстових корпусах. У даній роботі як основу системи детекції використовується модель mBERT з трьома різними класифікаційними головами.

Архітектура трансформера та механізм самоуваги

Трансформер - це архітектура нейронної мережі, запропонована Vaswani et al. (2017) [12], що повністю відмовляється від рекурентності та згорткових операцій на користь механізму самоуваги (self-attention). Ключова ідея полягає в тому, що при обробці кожного токена послідовності модель зважено враховує всі інші токени одночасно, а не лише попередній стан як у рекурентних мережах.

Механізм самоуваги обчислює для кожного токена три вектори: запит (Query), ключ (Key) та значення (Value). Вага впливу одного токена на інший визначається скалярним добутком їхніх векторів запиту та ключа, нормованим через функцію *softmax*. Такий підхід дозволяє моделі безпосередньо пов'язувати семантично пов'язані слова незалежно від відстані між ними у тексті.

Повна архітектура трансформерного енкодера включає декілька послідовних блоків, кожен з яких містить шар багатоголової уваги (Multi-Head Attention) та прошарок прямого поширення (Feed-Forward Network). Між підшарами застосовуються залишкові з'єднання (residual connections) та нормалізація (Layer Normalization), що забезпечує стабільність навчання глибоких мереж.

Концепція багатоголової уваги (Multi-Head Attention) передбачає паралельне застосування декількох незалежних механізмів уваги з різними навченими проєкційними матрицями. Кожна «голова» вчиться звертати увагу на різні аспекти тексту: синтаксичні залежності, семантичну схожість, кореференцію тощо. Результати всіх голів конкатенуються та лінійно проєктуються у фінальне представлення.

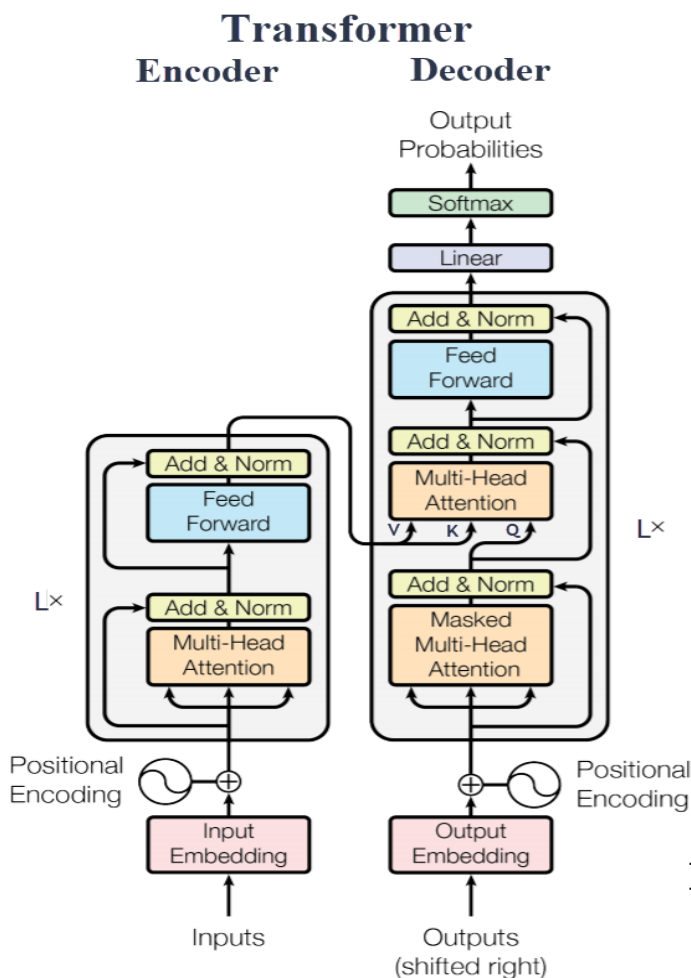


Рисунок 2.1 - Архітектура трансформерного енкодера (Vaswani et al., 2017)

[12]

Модель BERT та мультимовне розширення mBERT

BERT (Bidirectional Encoder Representations from Transformers) - модель попереднього навчання, запропонована Devlin et al. (2019) [13], що на момент публікації встановила нові рекорди на одинадцяти задачах обробки природної мови. Принципова відмінність BERT від попередніх підходів - двонаправлене врахування контексту: модель одночасно обробляє лівий і правий контекст кожного токена, а не послідовно зліва направо.

BERT навчається на двох задачах. По-перше, маскована мовна модель (Masked Language Modeling, MLM): у вхідному тексті випадково маскується 15% tokenів, і модель навчається їх передбачати на основі двобічного контексту. По-друге, передбачення наступного речення (Next Sentence Prediction, NSP): модель

класифікує, чи є два речення послідовними у тексті. Ці задачі дозволяють моделі засвоїти глибокі мовні патерни без розмітки даних.

Архітектура моделі bert-base містить 12 блоків трансформерного енкодера, прихований розмір $H = 768$ та 12 голів уваги. Кожен блок послідовно застосовує багатоголовий self-attention та двошаровий feed-forward network із функцією активації GELU. Загальна кількість параметрів становить близько 110 мільйонів.

Токенізація тексту у BERT здійснюється за алгоритмом WordPiece: рідкісні слова розбиваються на частіші підслова, що забезпечує фіксований розмір словника (~30 000 токенів для англomовної версії) при збереженні здатності обробляти довільну лексику. На початок кожної послідовності додається спеціальний токен [CLS] (classification), прихований стан якого після проходження через усі блоки енкодера агрегує контекстуальну інформацію всього тексту і традиційно використовується як вхід класифікатора.

mBERT (bert-base-multilingual-cased) є багатомовним розширенням базової моделі, навченим на корпусах 104 мов, включаючи українську. Словник розширено до 119 547 токенів для охоплення різних алфавітів та морфологічних систем. Незважаючи на відсутність явного крос-лінгвального навчання, mBERT демонструє здатність до zero-shot cross-lingual transfer - перенесення знань між мовами, - що було верифіковано дослідженнями Pires et al. (2019) [14]. Це робить mBERT придатним для роботи з україномовними текстами без потреби у великому мовно-специфічному корпусі.

Трансферне навчання та стратегія заморожування енкодера

Трансферне навчання (transfer learning) - підхід, за якого модель, попередньо навчена на великому загальному корпусі, адаптується до вузької цільової задачі. Для трансформерних моделей виділяють дві основні стратегії.

Перша - повне донавчання (full fine-tuning): всі параметри моделі, включаючи ваги базового енкодера, оновлюються на цільових даних. Ця стратегія дає найкращі результати при достатньому обсязі розміченого датасету, однак вимагає значних обчислювальних ресурсів і ризикує катастрофічним забуванням (catastrophic forgetting) - деградацією загальних мовних знань при навчанні на вузькій задачі.

Друга - екстракція ознак (feature extraction): ваги базового енкодера заморожуються, а навчання обмежується лише класифікаційним шаром. Ця стратегія обчислювально ефективна, захищає мовні знання від деградації та є доцільною при відносно малих датасетах.

У даній роботі застосована стратегія повного донавчання: всі 177 мільйонів параметрів моделі, включаючи ваги базового mBERT-енкодера та класифікаційного шару, оновлювались під час навчання. Така стратегія є обґрунтованою з огляду на розмір датасету, та забезпечує максимальну адаптацію мовних представлень до специфіки україномовних фейкових новин. Порівняльне дослідження трьох різних архітектур класифікаційних голів при однакових умовах навчання дозволяє ізолювати вплив топології верхніх шарів від впливу самого енкодера.

Класифікаційні голови: теоретичні основи

Класифікаційна голова (classification head) - це нейромережевий модуль, розміщений поверх базового енкодера, завданням якого є перетворення векторного представлення тексту на передбачення класу. Архітектура голови визначає, наскільки складні нелінійні залежності здатна моделювати система та як вона агрегує інформацію з векторного простору енкодера.

1. Лінійна класифікація

Найпростіший підхід - пряма лінійна проєкція вектора [CLS] у простір класів через один повнозв'язний шар. Перевага: мінімальна кількість параметрів і висока інтерпретованість. Обмеження: лінійна нерозподільна гіперплощина може бути недостатньою для складних семантичних патернів, характерних для маніпулятивних текстів. Для регуляризації та запобігання перенавчанню застосовується Dropout - техніка випадкового вимкнення нейронів під час навчання.

[15]

2. Багатошаровий перцептрон із пакетною нормалізацією

Розширення лінійного класифікатора шляхом додавання проміжного прихованого шару дозволяє моделі апроксимувати нелінійні рішачі межі. Ключовим елементом є Batch Normalization (BatchNorm) - техніка нормалізації активацій по міні-батчу з навченими параметрами масштабу та зміщення. [16] BatchNorm усуває ефект внутрішнього коваріаційного зсуву (internal covariate shift): стабілізує розподіл активацій між шарами у процесі навчання, що пришвидшує збіжність та дозволяє використовувати вищі значення швидкості навчання. Функція активації ReLU вносить нелінійність між шарами.

3. Пулінг на основі механізму уваги

Альтернативний підхід до агрегації інформації з тексту - відмова від токена [CLS] на користь зваженого усереднення прихованих станів усіх токенів. Механізм Attention Pooling навчає скалярний скор важливості для кожного токена; токени, що несуть більше маніпулятивне навантаження (емоційно забарвлені слова, сенсаційні фрази), отримують вищі ваги. Теоретично цей підхід є виразнішим, ніж використання єдиного агрегуючого токена, однак вимагає більшого обсягу даних для стабільного навчання параметрів механізму уваги. Замість BatchNorm тут застосовується Layer Normalization - нормалізація по вимірах ознак окремого зразка, стійка до малих розмірів батчу. [17]

Порівняльний аналіз цих трьох архітектур при однакових умовах навчання дозволяє емпірично встановити, наскільки ускладнення топології класифікаційного шару впливає на здатність системи детектувати фейкові новини в україномовному датасеті. Конкретна реалізація, гіперпараметри та результати порівняння описані у розділі 3.

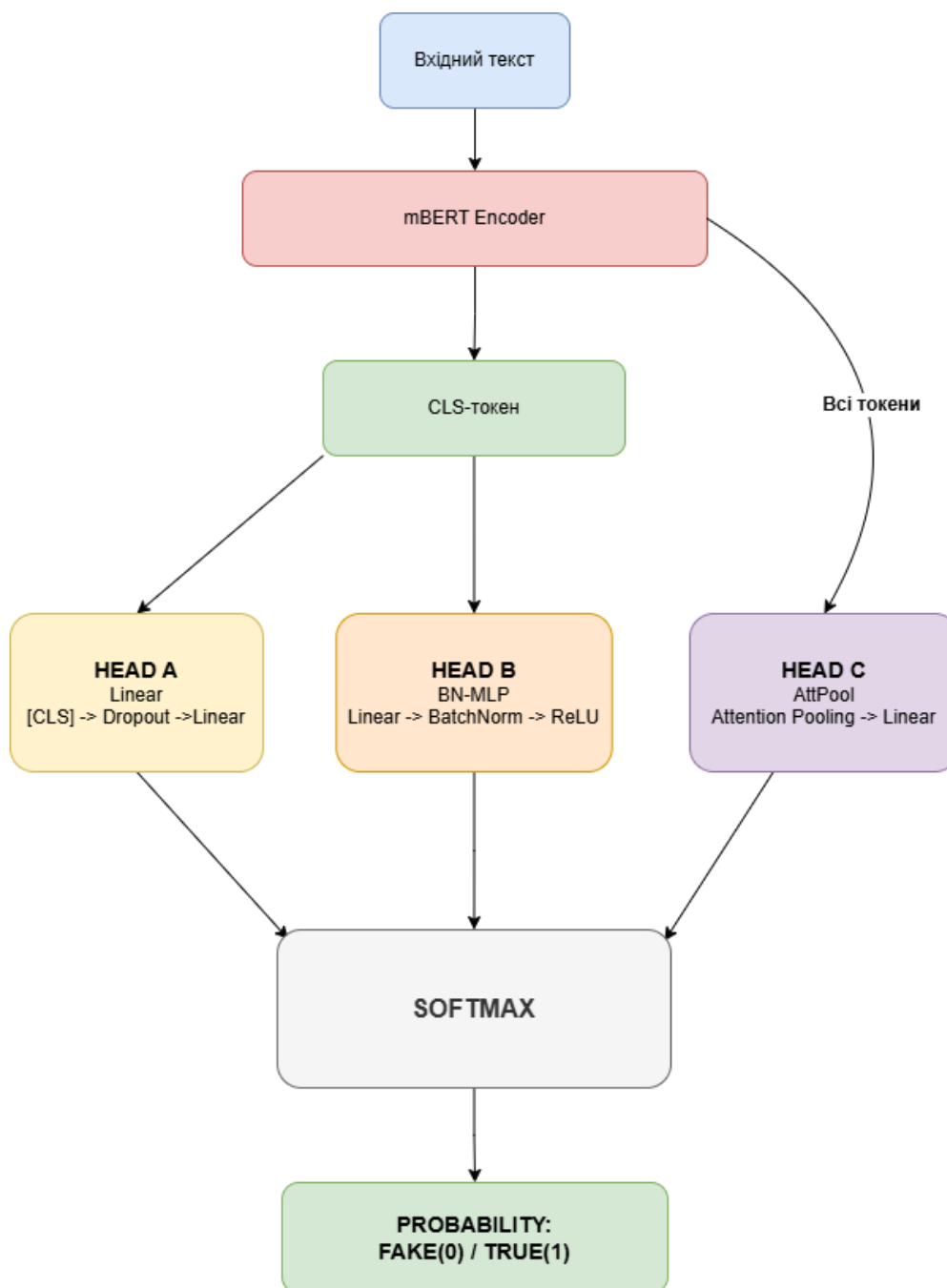


Рисунок 2.2 - Схема трьох досліджуваних архітектур класифікаційних голів
поверх mBERT енкодера

2.4 Формування датасету

Перший датасет призначений для задачі детекції фейкових новин і містить 21 738 зразків. Розподіл міток: 10 869 правдивих новин (мітка 1 - TRUE) та 10 869 фейкових новин (мітка 0 - FAKE). Датасет є збалансованим - що не потребує додаткової компенсації дисбалансу при навчанні.

Збір даних здійснювався з кількох джерел:

- Правдиві новини зібрано з верифікованих українських медіа: Укрінформ, Українська правда, Суспільне та офіційні прес-служби державних органів.
- Фейкові новини отримано з фактчекінгових порталів: StopFake.org (розбори російської дезінформації про Україну), VoxCheck (перевірка заяв українських політиків та публічних осіб), а також з розділу спростувань Укрінформу.

Кожен зразок у категорії фейкових новин є верифікованим - тобто інформацією, хибність якої підтверджена редакцією фактчекінгового порталу.

Розбиття датасету здійснювалось по-різному залежно від моделі. Для класичних моделей (логістична регресія, SVM) застосовано співвідношення 80/20: 80% зразків використовувались для навчання, 20% - для фінального тестування. Для моделі на основі BERT застосовано тричастинне розбиття: 70% - тренувальна вибірка (15 216 зразків), 15% - валідаційна вибірка (3 261 зразків) для моніторингу якості під час навчання, 15% - тестова вибірка (3 261 зразків) для фінальної оцінки.

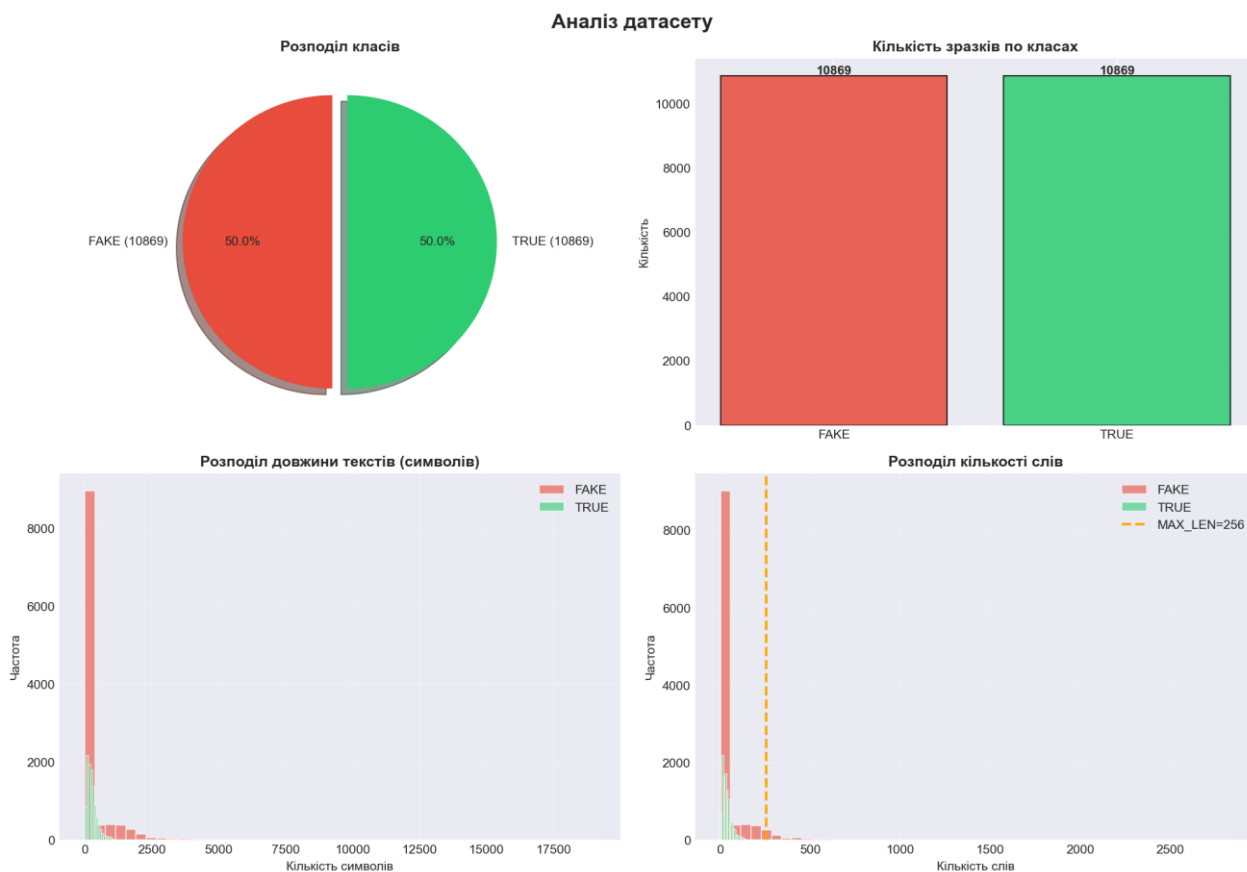


Рисунок 2.3 - Аналіз датасету фейкових та правдивих новин

Саме тому для розв'язання задачі класифікації доцільно застосовувати архітектури на основі трансформерів, зокрема модель mBERT (Multilingual BERT). На відміну від класичних підходів, BERT використовує механізм двонаправленої уваги (Self-Attention), що дозволяє моделі аналізувати не лише окремі слова чи біграми, а й складні семантичні залежності між ними. Це критично важливо для української мови, де тематична приналежність тексту часто визначається не наявністю конкретного терміна, а загальним контекстом речення та синтаксичними зв'язками між загальноновживаними словами.

2.5 Програмний інструментарій та засоби розробки

Програмна реалізація системи була виконана мовою програмування Python, яка є сучасним стандартом у галузі машинного навчання та обробки природної мови (NLP).

- Для завантаження, структурування та пакетної обробки наборів даних у форматі .csv використано бібліотеку Pandas.
- Математичний апарат класичного машинного навчання імплементовано за допомогою бібліотеки Scikit-learn. З її допомогою реалізовано векторизацію (TfidfVectorizer), навчання моделей (LogisticRegression, LinearSVC) та обчислення метрик точності.
- Для проведення морфологічного аналізу україномовних текстів та процесу лематизації використано сучасний аналізатор PyMorphy3, що дозволило значно зменшити розмірність простору ознак.
- Для реалізації архітектури BERT та процесу навчання використано бібліотеку PyTorch та Hugging Face Transformers, що надає готові реалізації моделі bert-base-multilingual-cased з токенизатором.
- Для візуалізації процесу навчання та аналізу датасету використано бібліотеки Matplotlib та Seaborn.
- Для генерації хмар слів використано бібліотеку WordCloud.

Таким чином, у даному розділі розглянуто теоретичні основи використаних методів та алгоритмів: математичний апарат TF-IDF векторизації, принципи роботи

логістичної регресії та SVM, архітектуру трансформера і моделі BERT, а також описано сформовані датасети та програмний інструментарій. Отримані теоретичні знання є основою для практичної реалізації системи, описаної у розділі 3.

3 РЕАЛІЗАЦІЯ СИСТЕМИ ТА РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ

3.1 Базова модель детекції фейкових новин (Baseline)

Перший етап практичної реалізації передбачав побудову базової моделі (Baseline) на основі логістичної регресії з TF-IDF векторизацією. Мета цього кроку - встановити орієнтирні метрики якості, відносно яких у подальшому оцінюватимуться складніші архітектури на базі нейронних мереж-трансформерів (зокрема, mBERT).

Реалізація пайплайну

Реалізацію виконано мовою програмування Python із використанням бібліотеки Scikit-learn. Конвеєр обробки (пайплайн) складається з трьох послідовних етапів: попередньої очистки тексту, перетворення текстових даних у числові вектори (TF-IDF) та безпосереднього навчання класифікатора.

1. Попередня обробка: очищення текстів (видалення URL-адрес, спецсимволів, стоп-слів) та лематизація за допомогою бібліотеки PyMorphu3. Зведення слів до початкової форми суттєво зменшило розмірність простору ознак.

```
def preprocess_text(text):  
    if pd.isna(text):  
        return ""  
  
    text = str(text).lower()  
    text = re.sub(r'^а-яіієз\s|$', '', text)  
    tokens = text.split()  
    lemmatized = []  
    for token in tokens:  
        if len(token) > 2 and token not in STOPWORDS:  
            parsed = morph.parse(token)[0]  
            lemma = parsed.normal_form  
            lemmatized.append(lemma)  
    return ' '.join(lemmatized)
```


2. Векторизація тексту: для перетворення тексту у вектори застосовано `TfidfVectorizer`. Для оптимізації обчислень словник було обмежено до 30 000 найбільш інформативних ознак (`max_features=30000`). Алгоритм враховує як окремі слова, так і біграми (`ngram_range=(1, 2)`). Для фільтрації корпусного шуму відкинуто аномально рідкісні та надмірно часті слова (`min_df=2`, `max_df=0.9`), а логарифмічне масштабування (`sublinear_tf=True`) дозволило згладити вплив найуживаніших термінів.
3. Класифікація: базовим класифікатором виступає оптимізована модель `LogisticRegression`. Використано оптимізатор `saga`, який є найбільш ефективним для розріджених текстових матриць. Зменшена сила регуляризації (`C=2.0`, `penalty='l2'`) покращила адаптацію моделі до складних даних, а автоматичне балансування ваг (`class_weight='balanced'`) дозволило нівелювати можливий дисбаланс класів у вибірці.

```
vectorizer = TfidfVectorizer(
    max_features=30000,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.9,
    sublinear_tf=True,
    use_idf=True,
    smooth_idf=True
)

model = LogisticRegression(
    C=2.0,
    penalty='l2',
    solver='saga',
    class_weight='balanced',
    random_state=42,
    max_iter=2000,
    tol=1e-4,
    n_jobs=-1,
    verbose=1
)
```

Результати базової моделі

Після завершення процесу навчання оптимізованої логістичної регресії було проведено оцінку її ефективності на відкладеній тестовій вибірці. Отримані результати зведено у таблицю 3.1.

Модель	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline LogReg	0.8408	0.8541	0.8219	0.8377	0.9216

Таблиця 3.1 - Метрики якості базової моделі (LogReg + TF-IDF)

Завдяки оптимізації гіперпараметрів базова модель продемонструвала загальну точність (Accuracy) на рівні **84.08%** та збалансовану метрику **F1-score 83.77%**. Показник Precision (85.41%) дещо перевищує Recall (82.19%). Це означає,

що класифікатор має незначну схильність до «перестраховки»: він рідше помиляється, коли маркує новину як фейкову (низький відсоток хибнопозитивних спрацьовувань), однак пропускає близько 17.8% реальних фейків, класифікуючи їх як правдиві. Детальний розподіл правильних та хибних передбачень візуалізовано за допомогою матриці помилок (Confusion Matrix) на рисунку 3.1.

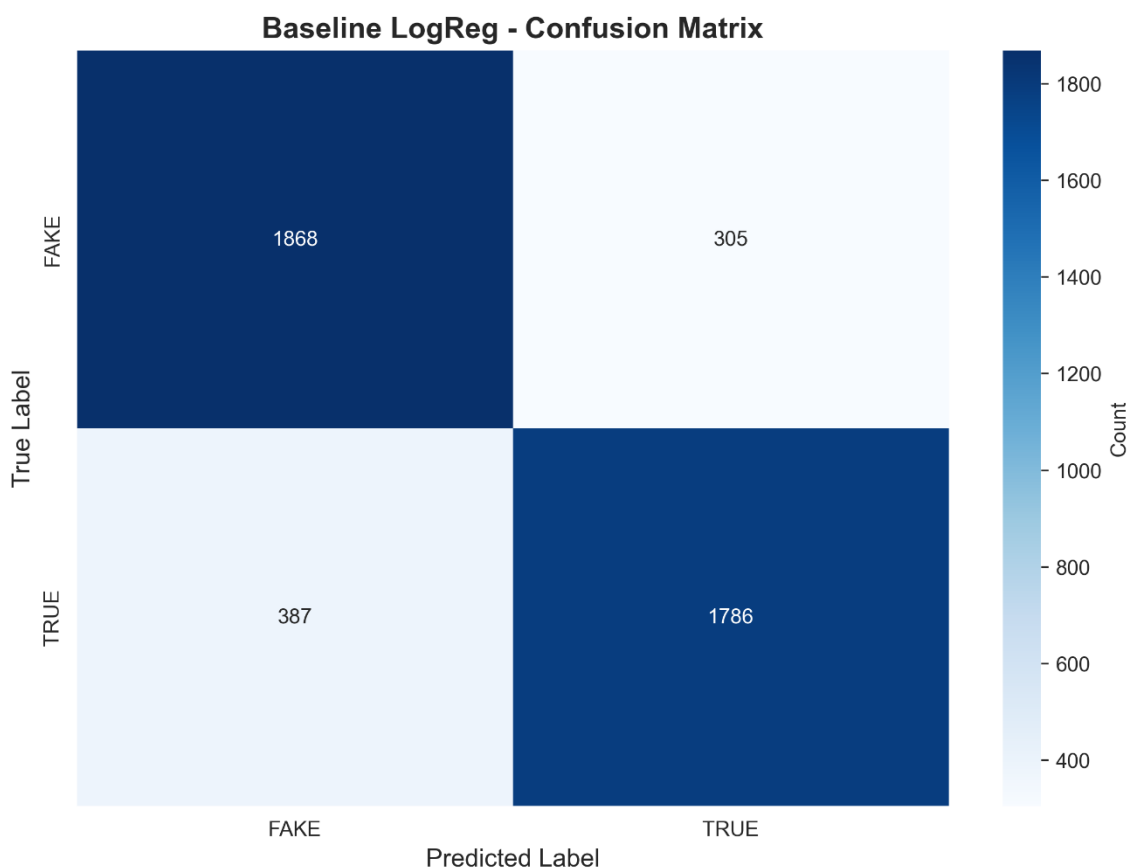


Рисунок 3.1 - Матриця помилок базової моделі логістичної регресії

Високе значення площі під кривою помилок ($\text{ROC-AUC} = 0.9216$) свідчить про відмінну здатність моделі ранжувати класи за ймовірнісними оцінками. Візуалізацію кривої (ROC Curve), яка демонструє співвідношення частки істинно позитивних до хибнопозитивних класифікацій при зміні порогу відсікання, наведено нижче на рисунку 3.2. Однією з ключових переваг лінійних моделей, таких як логістична регресія, є їхня висока інтерпретованість. Аналіз вагових коефіцієнтів моделі (Рисунок 3.3) дозволяє виявити найбільш впливові лексичні ознаки, на які спирається алгоритм під час прийняття рішень.

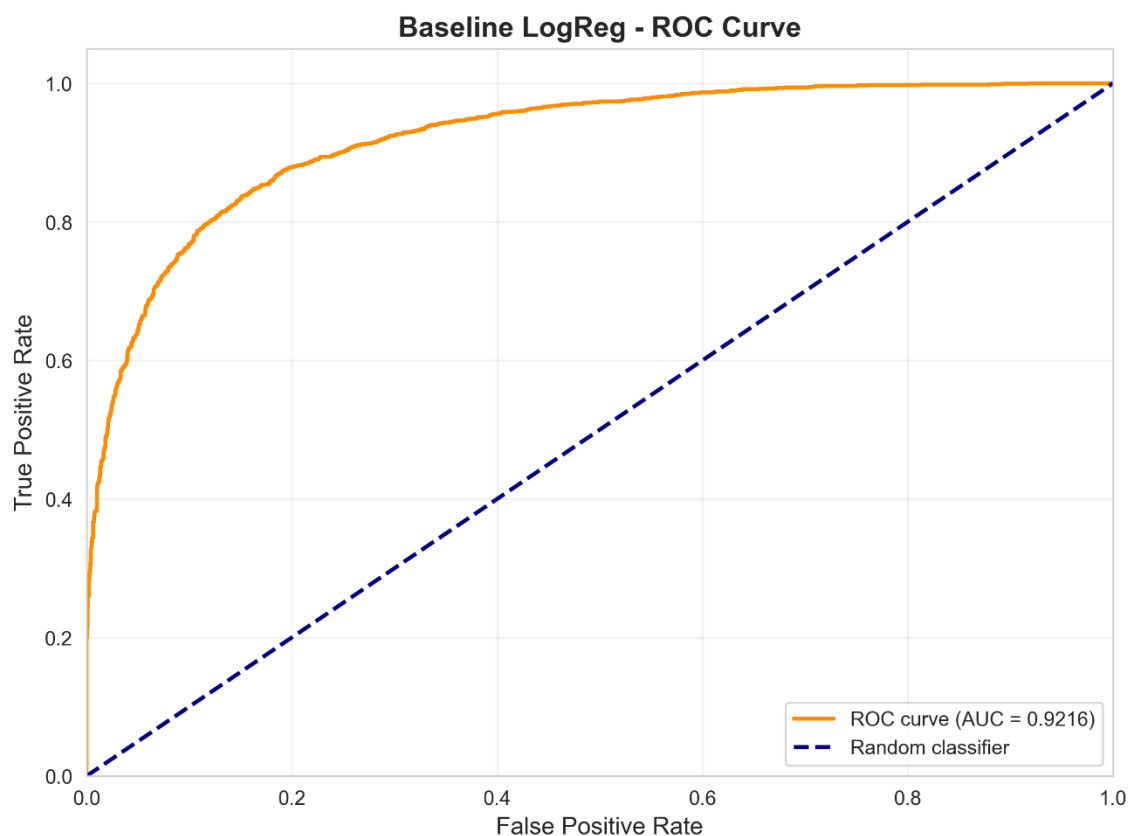


Рисунок 3.2 - ROC-крива базової моделі детекції фейків

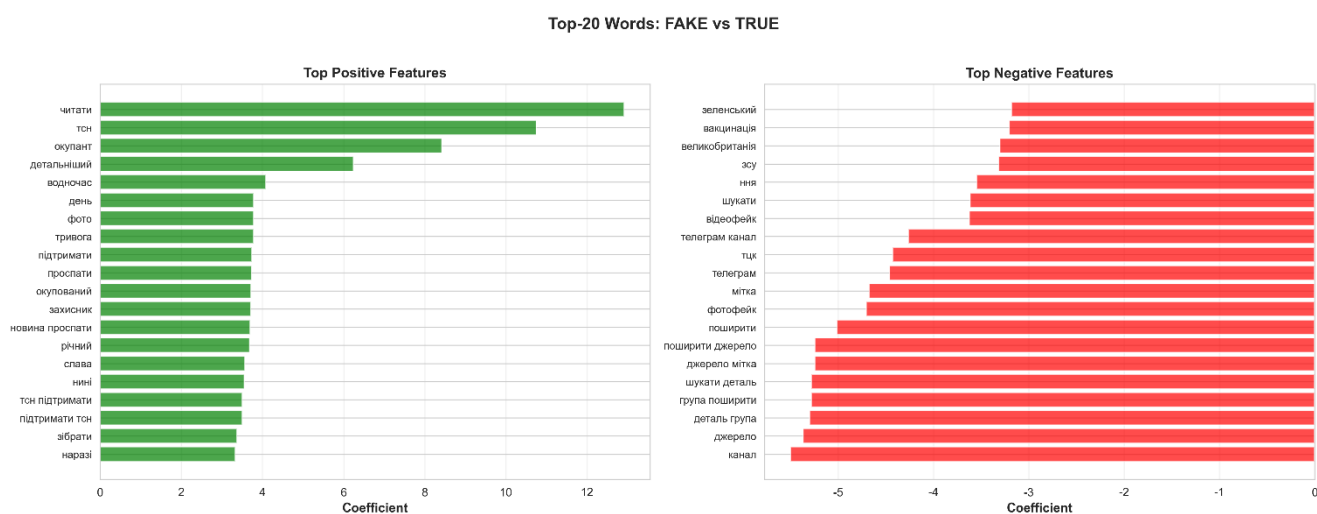


Рисунок 3.3 - Найбільш вагомі лексичні маркери для класів TRUE та FAKE

Як видно з Рисунка 3.3, аналіз вагових коефіцієнтів виявив специфічні зміщення у навчальній вибірці. Для ідентифікації класу FAKE лінійна модель спиралася переважно на маркери каналів поширення («телеграм канал», «група поширити»), тимчасові теми («вакцинація») та специфічну лексику фактчекерів («відеофейк»). Натомість для класу TRUE найвагомішими виявилися не ознаки достовірності, а брендування конкретних українських медіа (зокрема, «ТСН»,

«Читати», «детальніший») та типова лексика воєнних зведень («окупант», «захисник»).

Цей результат наочно демонструє головний недолік парадигми TF-IDF у задачі детекції фейків: алгоритм орієнтується на специфічні теги джерел чи популярні слова, повністю ігноруючи контекст і суть маніпуляції. Дане обмеження фундаментально обґрунтовує необхідність використання трансформерних архітектур (mBERT), здатних аналізувати глибинну семантику тексту незалежно від наявності конкретних назв медіа.

Висновки щодо базової моделі:

Проведені експерименти підтвердили, що TF-IDF-представлення текстів у поєднанні з логістичною регресією забезпечує достатньо високий базовий рівень (близько 84% точності) для відокремлення фейкових новин від правдивих. Однак фундаментальне обмеження парадигми «мішка слів» (Bag-of-Words) полягає в ігноруванні порядку слів, синтаксичних зв'язків та глибинного контексту. Модель не здатна розпізнати складні маніпулятивні конструкції, якщо вони складаються із загальноновживаних «нейтральних» слів. Саме це обмеження зумовлює необхідність переходу до сучасних архітектур на базі трансформерів, здатних захоплювати семантику тексту.

3.2 Детекція фейкових новин на основі mBERT

Другим етапом практичної реалізації стало дослідження різних архітектур класифікаційних голів (Classification Heads) поверх базової моделі bert-base-multilingual-cased (mBERT). Головна мета цього етапу - з'ясувати, наскільки ускладнення верхніх шарів нейромережі впливає на її здатність виявляти маніпулятивний контент та розпізнавати семантику фейків.

Спільні гіперпараметри навчання

Для забезпечення об'єктивності порівняння результатів, усі три досліджувані архітектури навчалися за абсолютно ідентичних умов. Було застосовано стратегію повного донавчання (full fine-tuning), за якої в процесі зворотного поширення помилки оновлювалися ваги як самого mBERT-енкодера, так і нових класифікаційних шарів.

У ході експериментів було встановлено наступні оптимальні гіперпараметри:

- Максимальна довжина послідовності (`max_len`): 256 токенів. На відміну від стандартних 128 токенів, збільшене вікно дозволяє моделі захоплювати ширший контекст новинних статей, що є критично важливим для виявлення складних маніпуляцій.
- Розмір міні-батчу (`batch_size`): 16
- Кількість епох (`epochs`): 3.
- Швидкість навчання (`learning_rate`): 1×10^{-5} . Дещо знижений показник LR забезпечує стабільну збіжність (`convergence`) моделі та запобігає катастрофічному забуванню попередньо вивчених мовних патернів.
- Регуляризація: використано оптимізатор AdamW із ваговим розпадом `weight_decay = 0.01`.
- Коефіцієнт Dropout було збільшено до 0.4 для ефективнішої боротьби з перенавчанням (`overfitting`) на специфічних назвах медіа та тимчасових інфоприводах. Крім того, застосовано лінійний прогрів швидкості навчання (`warmup`) на перших 10% кроків.

Процес навчання безперервно оцінювався за метрикою F1-score на валідаційній вибірці з використанням механізму ранньої зупинки (Early Stopping) для фіксації найкращих ваг моделі.

Архітектури класифікаційних голів

Для агрегації ознак, видобутих mBERT, було реалізовано та досліджено три класифікаційні голови:

1. **Head A (Linear):** реалізує базову лінійну класифікацію. Вектор [CLS]-токена розмірністю 768 подається на вхід Dropout-шару ($p=0.4$), а потім на повнозв'язний лінійний шар, що проєктує вектор у простір двох класів: `Linear(768  $\rightarrow$  2)`. Головна перевага цієї архітектури - мінімальна кількість додаткових параметрів та висока стійкість до перенавчання (завдяки підвищеному показнику Dropout).

2. **Head B (MLP):** реалізує двошаровий перцептрон із пакетною нормалізацією. Вектор [CLS] подається на перший лінійний шар $\text{Linear}(768 \rightarrow 512)$, потім - на $\text{BatchNorm1d}(512)$ для стабілізації розподілу активацій, далі - на функцію активації ReLU, Dropout-шар ($p=0.4$) та фінальний лінійний шар $\text{Linear}(512 \rightarrow 2)$. Пакетна нормалізація пришвидшує збіжність та дозволяє моделі ефективно апроксимувати нелінійні рішачі межі.
3. **Head C (Attention Pooling):** реалізує класифікацію на основі механізму уваги. На відміну від попередніх варіантів, ця архітектура не покладається виключно на [CLS]-токен, а обробляє приховані стани всіх токенів послідовності. Навчений скалярний показник важливості (attention score) визначає внесок кожного слова у фінальне представлення. Після зваженого усереднення отриманий вектор послідовно проходить через $\text{Linear}(768 \rightarrow 256)$, LayerNorm, ReLU, Dropout ($p=0.4$) та фінальну проєкцію $\text{Linear}(256 \rightarrow 2)$.

Результати порівняльного аналізу архітектур

Після завершення навчання трьох моделей на збалансованому датасеті їхня ефективність була оцінена на відкладеній тестовій вибірці. Отримані результати зведено у таблицю 3.2.

Голова	Accuracy	Precision	Recall	F1	ROC-AUC
Head A (Linear)	0.9243	0.9250	0.9243	0.9242	0.9829
Head B (BN-MLP)	0.9270	0.9293	0.9270	0.9269	0.9823
Head C (AttPool)	0.9295	0.9306	0.9295	0.9294	0.9827

Таблиця 3.2 - Порівняння трьох класифікаційних голів mBERT на задачі детекції фейків

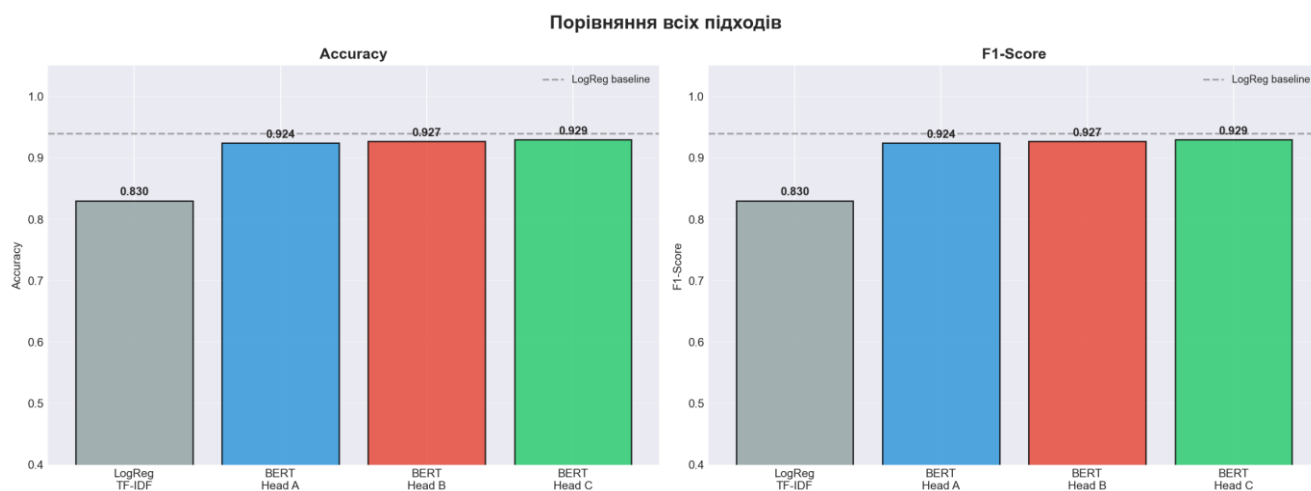


Рисунок 3.4 - Порівняння метрик якості для різних архітектур класифікаційних голів mBERT

Всі три нейромережеві архітектури суттєво перевершують базову модель логістичної регресії (Accuracy 84.08%) за всіма метриками. Найвищий показник **F1-score** демонструє складна архітектура **Head C (0.9294)** порівняно з 0.9242 у Head A та 0.9269 у Head B. Водночас найвище значення площі під кривою **ROC-AUC (0.9829)** досягнуто базовою архітектурою Head A, що свідчить про її відмінну здатність ймовірно ранжувати класи. Значення Precision та Recall у всіх трьох голів практично дорівнюють одне одному - це прямий наслідок збалансованості навчального датасету та симетричності функції втрат.

Варто зауважити, що Head C, попри використання складного механізму уваги та більшу кількість параметрів, продемонструвала найкращий результат, проте відрив від простіших голів є незначним (у межах 0.5 процентного пункту). Це доводить ключову тезу: основний вклад у якість розпізнавання фейків вносить сам mBERT-енкодер (завдяки потужним попередньо навченим ембедінгам), тоді як вибір топології класифікаційного шару виступає лише інструментом тонкого налаштування моделі.

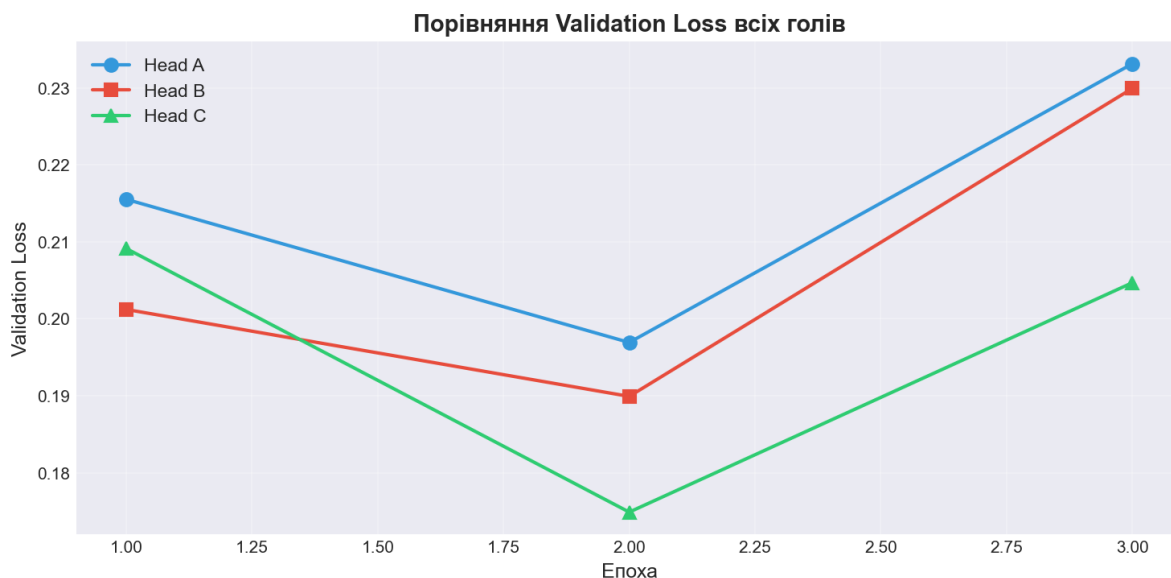


Рисунок 3.5 - Динаміка функції втрат архітектур mBERT

Графіки динаміки навчання для всіх трьох моделей підтверджують стабільне зниження функції втрат на валідації (Validation Loss) протягом трьох епох без ознак перенавчання (overfitting), що підтверджує коректність вибору швидкості навчання та коефіцієнта Dropout.

Аналіз матриць помилок (Confusion Matrices) виявив спільну тенденцію для всіх архітектур: їхньою найчастішою помилкою є хибне маркування правдивої новини як фейку (False Positive).

У контексті розробки автоматизованих систем фактчекінгу така поведінка моделі є цілком прийнятною: система діє за принципом "перестраховки", привертаючи увагу користувача до потенційно маніпулятивних, хоч і фактологічно достовірних текстів, мінімізуючи при цьому ризик пропустити реальну дезінформацію (False Negative).

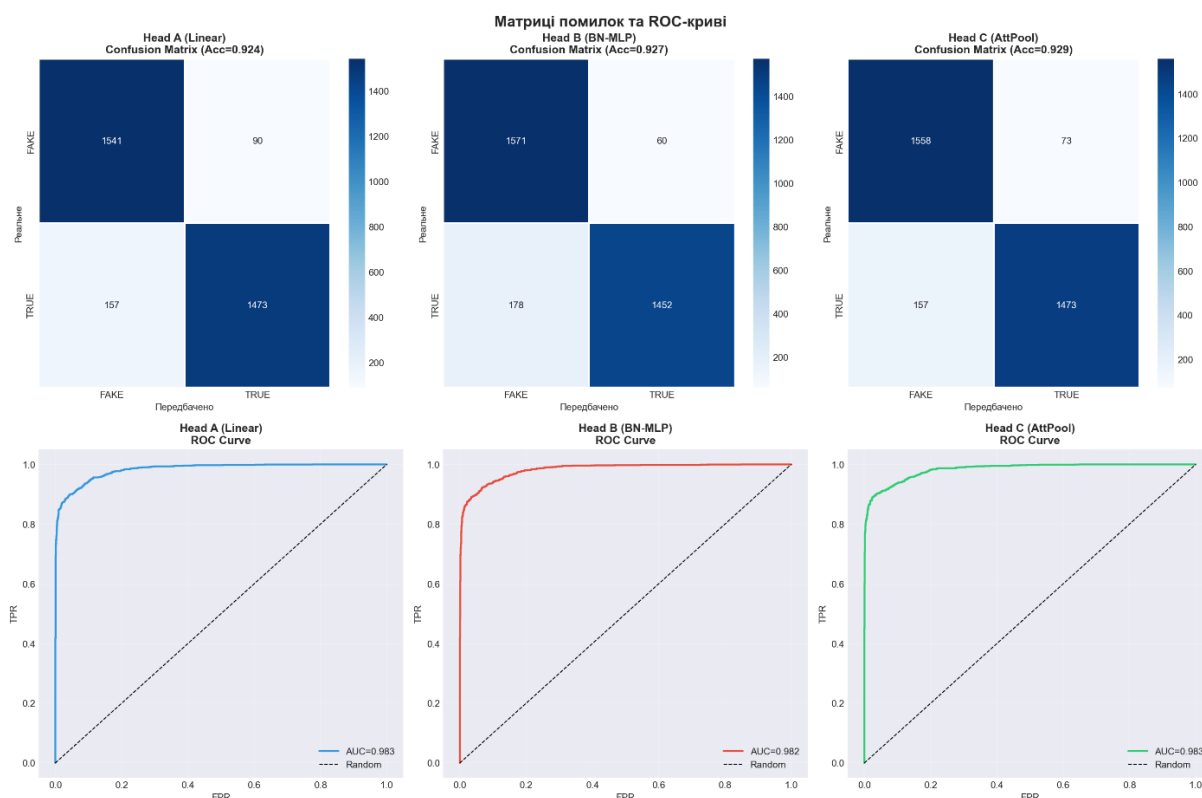


Рисунок 3.6 - Матриця помилок та ROC-криві для всіх архітектур

3.3 Експеримент А: Вплив дисбалансу класів на якість детекції фейків

Третій експеримент мав на меті дослідити стійкість розроблених неймережових архітектур до умов штучно створеного дисбалансу класів. У реальних бойових умовах інформаційного простору датасети новин майже ніколи не бувають ідеально збалансованими: фактчекінгові організації фізично здатні верифікувати лише невелику частку найгучніших фейків, тоді як загальний потік достовірного новинного контенту є в рази більшим. Тому критично важливо розуміти, як модель поводитиметься при навчанні на незбалансованих даних.

Умови проведення експерименту

Для моделювання реалістичного дисбалансу з оригінального датасету було взято 100% достовірних новин (10 869 зразків) та лише 50% фейкових новин (5 434 зразки), які були обрані випадково із фіксованим параметром `random_state=42`. Підсумковий незбалансований датасет (ExpA) склав 16303 тексти зі співвідношенням класів TRUE до FAKE приблизно 2:1.

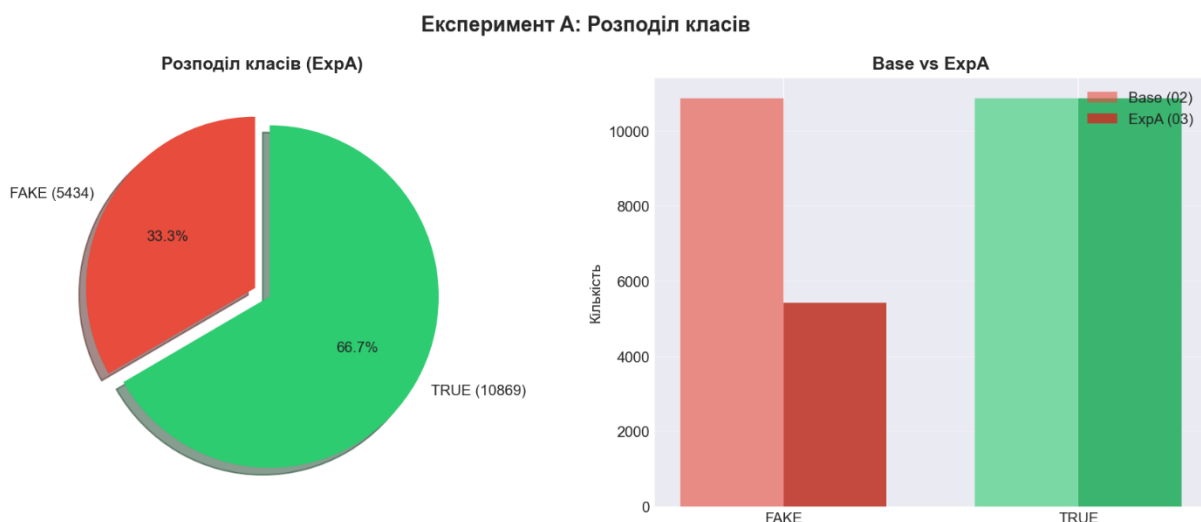


Рисунок 3.7 - Розподіл класів у незбалансованому датасеті (Експеримент А)

Аби ізолювати вплив саме дисбалансу класів, усі три архітектури класифікаційних голів, спільні гіперпараметри навчання (Dropout 0.4, LR 1e-5, AdamW) та стратегія розбиття датасету на вибірки залишилися абсолютно ідентичними до базового експерименту.

Результати класифікації на незбалансованих даних

Результати оцінки моделей, навчених на скороченому та незбалансованому датасеті, наведено у таблиці 3.3.

Голова	Accuracy	Precision	Recall	F1	ROC-AUC
Head A (ExpA)	0.9203	0.9211	0.9203	0.9206	0.9811
Head B (ExpA)	0.9272	0.9277	0.9272	0.9274	0.9807
Head C (ExpA)	0.9244	0.9245	0.9244	0.9244	0.9814

Таблиця 3.3 - Результати Експерименту А (дисбаланс TRUE:FAKE $\approx$ 2:1)

Результати Експерименту А демонструють відчутне, але зовсім не катастрофічне зниження загальної якості порівняно зі збалансованим датасетом. Найбільше зниження метрики F1-score зафіксовано для складної архітектури Head C (-0.0050) та базової Head A (-0.0037).

Цікавою аномалією стала поведінка архітектури **Head B (MLP з пакетною нормалізацією)**, яка продемонструвала найвищу стійкість до незбалансованих даних: її F1-score залишився на рівні 0.9274 (порівняно з 0.9269 у збалансованому варіанті). Це свідчить про те, що використання шару BatchNorm1d допомагає стабілізувати дисперсію ознак навіть тоді, коли один із класів домінує у вибірці.

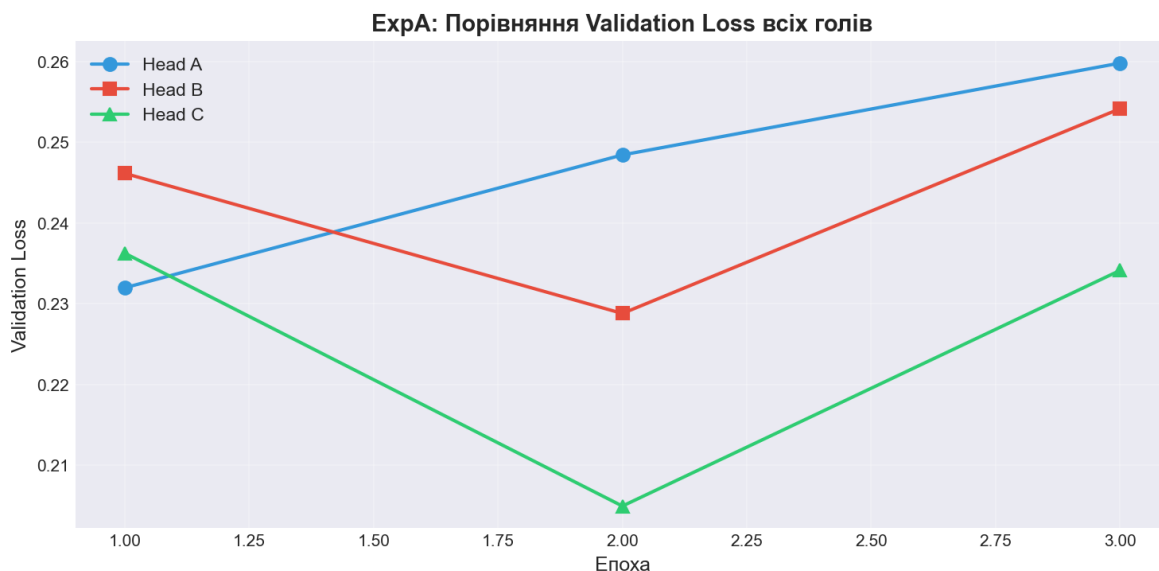


Рисунок 3.8 - Порівняння динаміки навчання трьох архітектур на незбалансованому датасеті

Аналіз значень ROC-AUC показує лише мікроскопічне зниження для всіх голів (у межах 0.002). Це підтверджує відому теоретичну тезу: метрика ROC-AUC є нечутливою до дисбалансу класів, оскільки оцінює загальну здатність моделі ранжувати ймовірності, на відміну від Accuracy та F1, які сильно залежать від встановленого порогу відсікання.

Підсумки експерименту: Загальна тенденція яскраво свідчить про те, що навіть при дворазовому дисбалансі класів і зменшенні навчальної вибірки (з 21 738 до 16 303 зразків), архітектури на базі mBERT зберігають високий рівень якості ($F1 > 92\%$). Ця стійкість забезпечується завдяки глибоким мовним представленням, набутим моделлю під час попереднього навчання (pre-training) на гігантських корпусах текстів. Модель не просто "зазубрює" частотні слова домінуючого класу, а продовжує спиратися на семантику та контекст маніпуляцій.

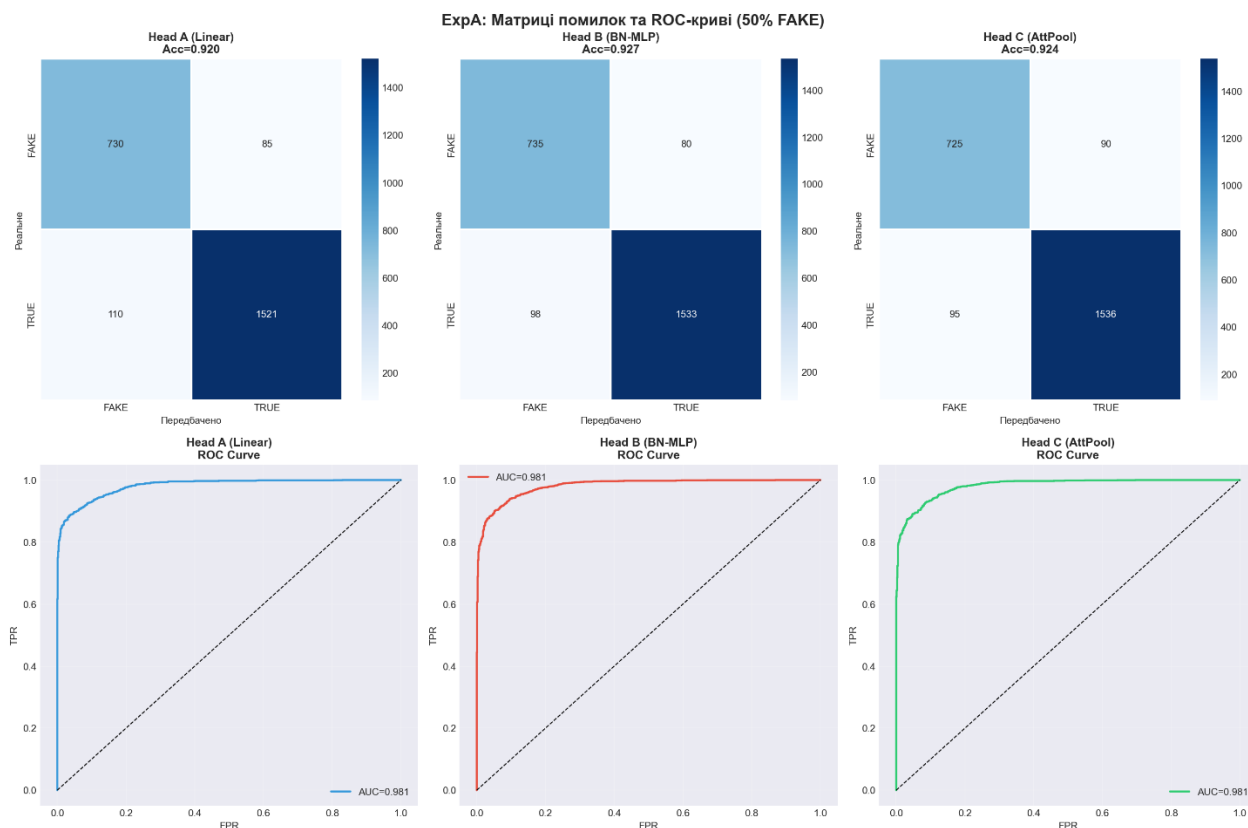


Рисунок 3.9 - Матриці помилок та ROC-криві для незбалансованого датасету

3.4 Базова модель тематичної класифікації (LinearSVC)

Паралельно з розробкою модуля детекції фейкових новин було реалізовано модуль тематичної класифікації. Для встановлення базових метрик (Baseline) застосовано метод опорних векторів з лінійним ядром (LinearSVC) у поєднанні з TF-IDF векторизацією.

Векторизація текстів здійснювалась за допомогою оптимізованого TfidfVectorizer. Словник було обмежено до 30 000 найбільш інформативних ознак (`max_features=30000`). Для врахування локального контексту використовувалися як уніграми, так і біграми (`ngram_range=(1,2)`). Фільтрація корпусного шуму налаштована дещо жорсткіше, ніж для бінарної класифікації: відкидалися рідкісні слова, що зустрічаються менше ніж у трьох документах (`min_df=3`), та надто загальні терміни, присутні у понад 85% текстів (`max_df=0.85`). Також застосовано логарифмічне згладжування (`sublinear_tf=True`) для зменшення впливу високочастотних слів.

Особливу увагу приділено налаштуванню самого класифікатора. Базовою моделлю виступив алгоритм LinearSVC з параметром `dual=False`. Використання прямої (primal), а не двоїстої (dual) задачі оптимізації є математично доцільним та обчислювально ефективним рішенням, оскільки кількість зразків у датасеті (50 000) перевищує кількість ознак у векторизаторі (30 000). Кількість ітерацій збільшено до 2000 для гарантованої збіжності градієнтного спуску.

Фундаментальною особливістю методу опорних векторів є те, що він видає лише відстань від об'єкта до розділяючої гіперплощини, а не ймовірнісні оцінки належності до класу. Оскільки розроблювана система (вебдодаток) вимагає виведення впевненості моделі у відсотках, базовий класифікатор було обгорнуто в мета-оцінювач `CalibratedClassifierCV`. Цей модуль виконує калібрування ймовірностей (Platt scaling) із застосуванням 3-кратої крос-валідації (`cv=3`). Такий підхід дозволив не лише отримати коректні ймовірнісні оцінки для кожної теми, але й підвищив загальну стійкість моделі до перенавчання (завдяки ансамблюванню на фолдах крос-валідації).

Результати базової моделі

Після навчання на датасеті з 5 категорій (бізнес, новини, політика, спорт, технології) модель була оцінена на тестовій вибірці. Загальна точність (Accuracy) склала **92.13%**, а збалансована метрика Macro F1 - **0.9209**.

Категорія	Precision	Recall	F1-Score
бізнес	0.93	0.93	0.9318
новини	0.86	0.85	0.8552
політика	0.88	0.89	0.8840
спорт	0.99	0.99	0.9880
технології	0.95	0.94	0.9456
Macro Avg	0.92	0.92	0.9209

Таблиця 3.4 - Метрики якості базової моделі LinearSVC

Аналіз метрик у розрізі окремих класів виявив значну диспропорцію. Найвищий показник зафіксовано для категорії «спорт» ($F1 = 0.9880$), що легко пояснюється вузькоспеціалізованою лексикою спортивних подій.

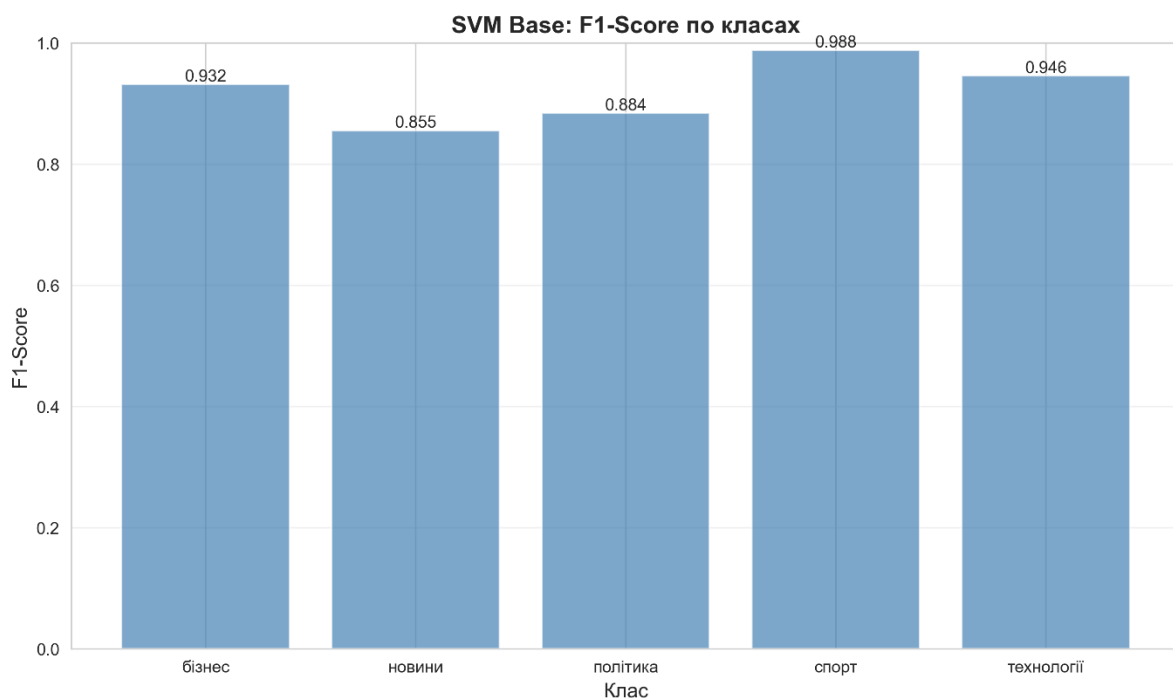


Рисунок 3.10 - Метрика F1-score для кожної тематичної категорії

Натомість найнижчі результати показали категорії «новини» (0.8552) та «політика» (0.8840). Це спонукало до глибшого аналізу помилок моделі за допомогою матриці плутанини, яка зображена на рисунку 3.11.

Матриця наочно виявила систематичну проблему: класифікатор масово плутає тексти з категорій «новини» та «політика». Це підтверджується детальним аналізом хибних передбачень.

Як видно з графіка помилок, який зображений на рисунку 3.12, алгоритм **135 разів** хибно класифікував «новини» як «політику» - це найбільше джерело похибки у всій системі. Дана проблема є концептуальною: категорія «новини» у зібраному корпусі є семантично розмитою (парасольковою) та охоплює загальні матеріали, які за своєю лексикою тісно перетинаються з внутрішньополітичними подіями.

Базова модель на основі TF-IDF, яка оперує виключно частотністю слів (Bag-of-Words) і не розуміє контексту, не здатна провести чітку межу між цими категоріями. Для вирішення цієї проблеми класифікацію повного датасету (5

класів) буде доручено контекстно-залежній нейромережі mBERT. Якщо навіть глибоке розуміння семантики не дозволить розв'язати цю колізію, буде розглянуто необхідність структурної оптимізації самого набору даних.

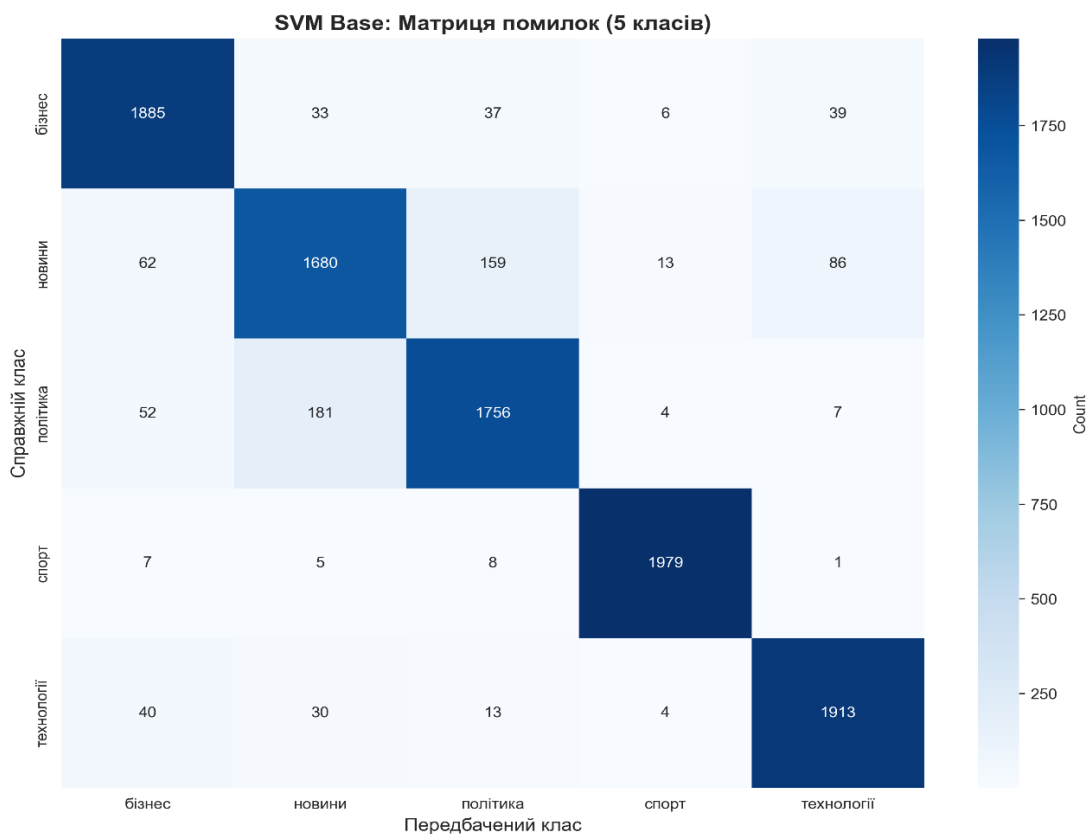


Рисунок 3.11 - Матриця помилок базової моделі LinearSVC

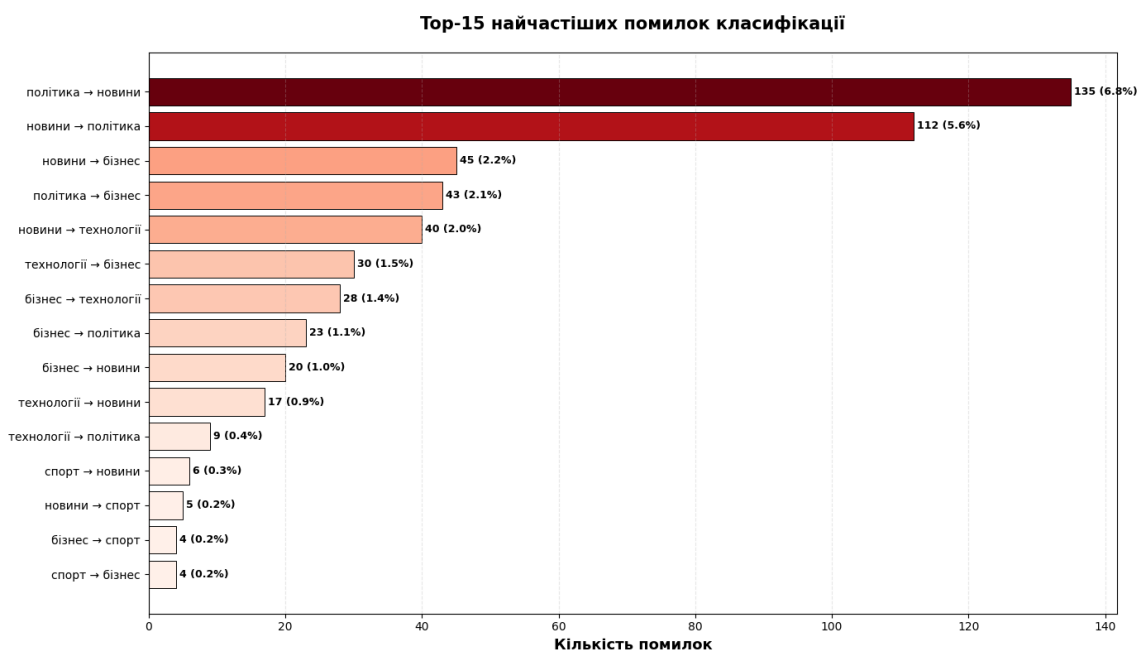


Рисунок 3.12 - Аналіз найбільш частих помилок між класами

3.5 Тематична класифікація на основі mBERT

Аналогічно до задачі детекції фейків, для тематичної класифікації було реалізовано та навчено неймережеву модель на базі багатомовного трансформера mBERT. Головна мета цього етапу -- визначити, чи дають глибокі контекстуальні представлення трансформера суттєвий приріст якості порівняно з класичним підходом (TF-IDF + LinearSVC) для задачі розпізнавання новинних рубрик.

Архітектура та гіперпараметри

Для тематичної класифікації застосовано архітектуру Head A (лінійна класифікаційна голова), яка зарекомендувала себе як найбільш стабільна та стійка до перенавчання конфігурація: вектор токена [CLS] подається на шар Dropout(0.4), а потім на лінійний шар Linear(768 $\rightarrow$ N_classes).

Для базового експерименту кількість класів N_classes = 5, для оптимізованого Експерименту В N_classes = 4. Гіперпараметри були використані на основі попередніх випробувань.

Результати базової моделі (5 класів) та проблематика даних

На першому етапі модель навчалася на повному датасеті з п'ятьма категоріями. Базова неймережева модель (BERT Topic Base) досягла загальної точності **94.59%** та F1-score **0.9459**, зображено на рисунку 3.13. Це краще за показники лінійної моделі SVM (92.13%), проте детальний аналіз результатів виявив збереження критичної проблеми.

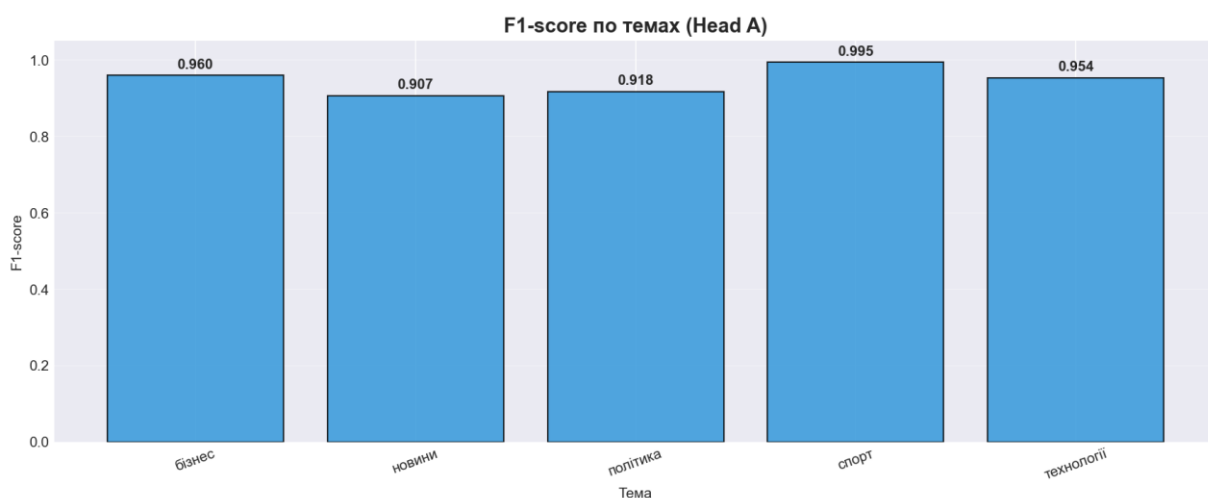


Рисунок 3.13 – F1-Score базової моделі mBERT (5 класів)

Як видно з матриці помилок, зображеної на рисунку 3.14, незважаючи на здатність трансформера розуміти складний мовний контекст, модель все одно продовжувала систематично плутати тексти з категорій «новини» та «політика». Це емпірично доводить ключову тезу: проблема полягає не у слабкості алгоритмів машинного навчання, а в самій природі даних. Категорія «новини» у зібраному датасеті виявилася «парасольковою» (семантично розмитою) - вона містить загальні сюжети, які лексично і контекстуально дублюють політичні події. Оскільки об'єктивної межі між цими класами в текстах не існує, будь-яка модель буде генерувати системну похибку.

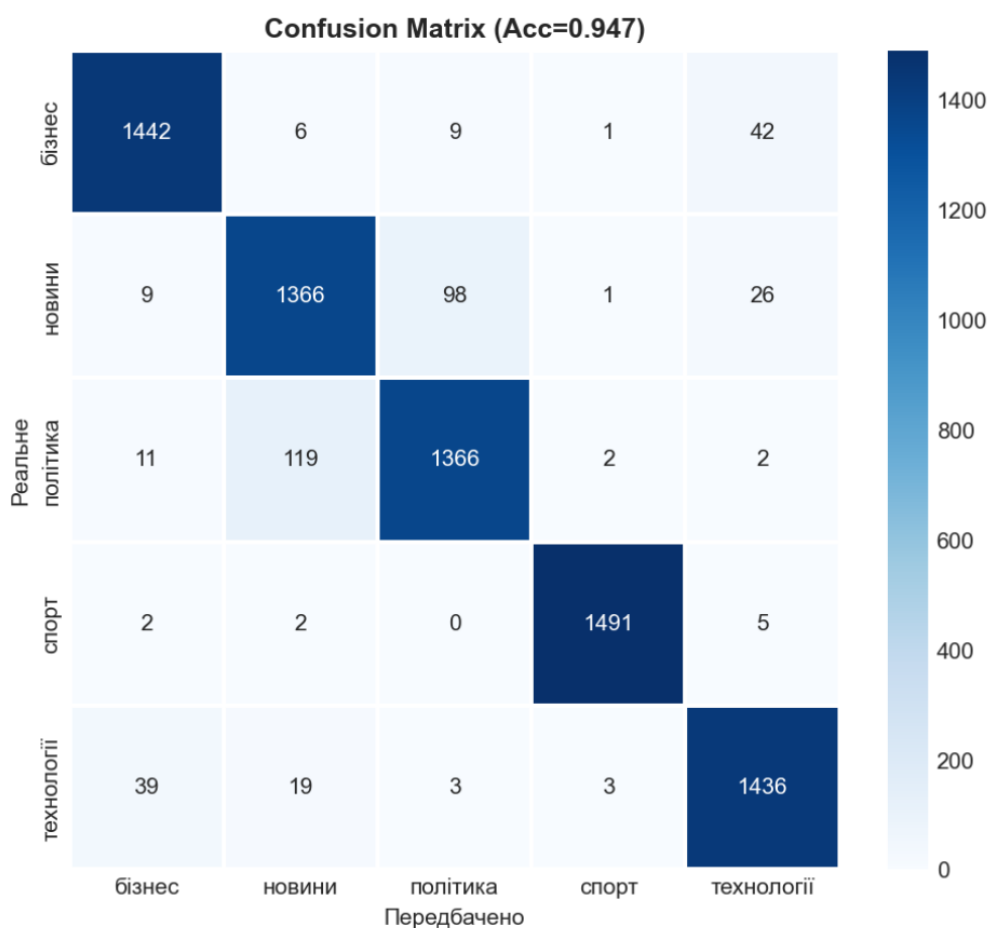


Рисунок 3.14 - Матриця помилок базової моделі mBERT (5 класів)

Детальний розподіл хибних класифікацій представлено на графіку 3.15 нижче.

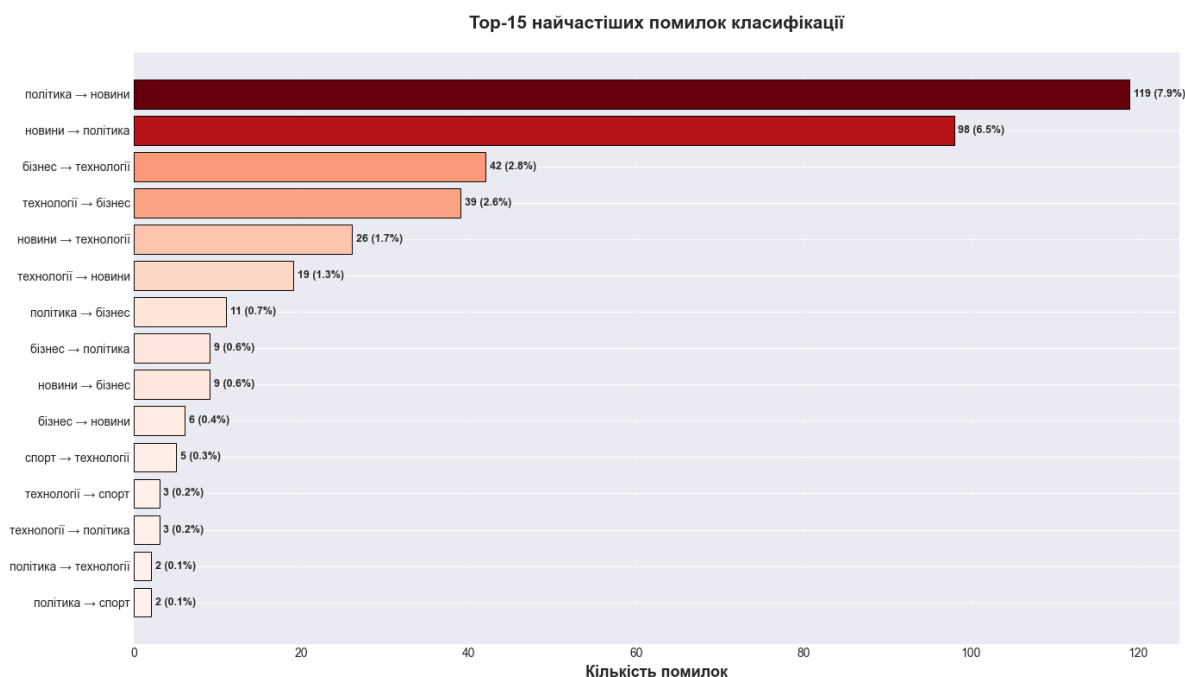


Рисунок 3.15 - Аналіз найбільш частих хибних передбачень

Як свідчать дані з графіка аналізу помилок, абсолютна більшість хибних передбачень концентрується виключно навколо цієї пари класів. Зокрема, алгоритм **119 разів** помилково відніс тексти з категорії «політика» до рубрики «новини», і дзеркально - **98 разів** класифікував загальні «новини» як «політику». Сумарно ці взаємні помилки (217 випадків) становлять лівову частку всіх неточностей моделі на тестовій вибірці. Це доводить, що навіть потужні механізми контекстуальної уваги трансформера mBERT не здатні провести межу там, де її семантично не існує: обидві рубрики оперують однаковими сутностями (назви державних органів, прізвища посадовців, суспільні події).

Цей висновок змусив вдаватися до структурної оптимізації та повністю видалити категорію «новини» з датасету. Цей процес цілеспрямованого очищення простору класів описано в Експерименті В.

Експеримент В: класифікація на оптимізованому датасеті (4 класи)

В рамках Експерименту В модель mBERT була перенавчена на очищеному датасеті, з якого повністю вилучено семантично розмиту категорію «новини». Оновлений набір даних складається з чотирьох чітко розмежованих предметних областей: «бізнес», «політика», «спорт» та «технології».

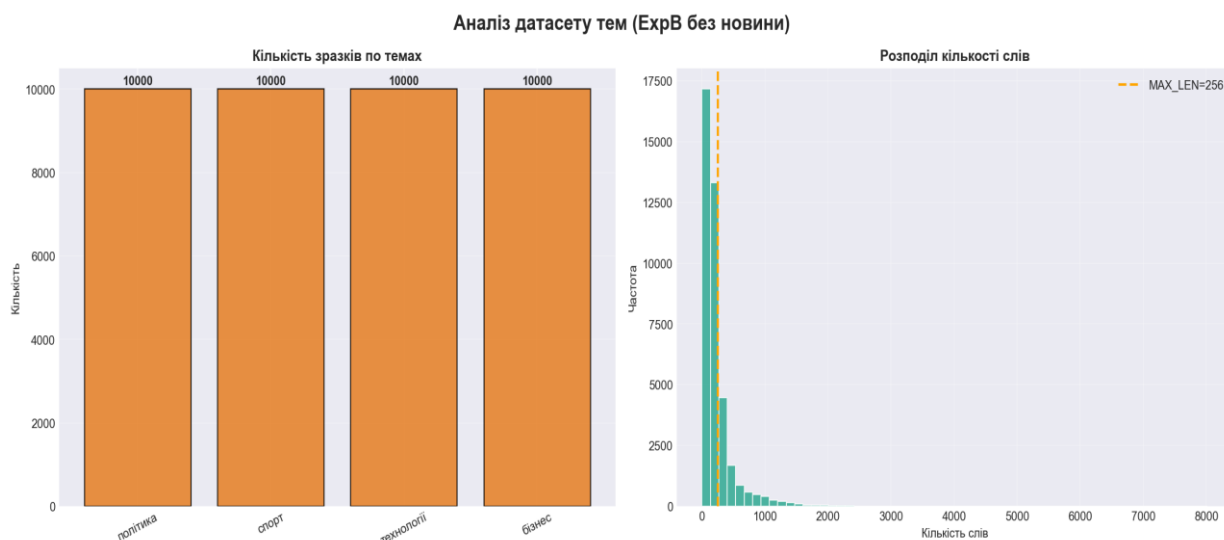


Рисунок 3.15 - Розподіл класів у датасеті після структурної оптимізації

Для цього експерименту використовувалася архітектура Head A (лінійна класифікаційна голова) з гіперпараметрами, зафіксованими у конфігурації навчання попередніх експериментів.

Аналіз результатів Експерименту В

Навчання та оцінка обох конфігурацій mBERT (на 5 та на 4 класи) проводилися за ідентичних умов для забезпечення об'єктивності порівняння. Отримані результати зведено у таблицю 3.6.

Конфігурація	Accuracy	Macro F1	Precision	ROC-AUC	Примітка
BERT Topic Base	0.9459	0.9459	0.9460	0.9949	5 класів
BERT Topic ExpB	0.9772	0.9772	0.9772	0.9983	4 класи

Таблиця 3.6 - Результати mBERT для тематичної класифікації

Як видно з таблиці 3.6, перехід до Експерименту В (повне видалення розмитої категорії з датасету) кардинально покращив результати. Якість роботи моделі стрімко зросла: загальна точність (Accuracy) та збалансована метрика F1 досягли **97.72%**, а площа під кривою ROC-AUC наблизилася до ідеального максимуму - 0.9983.

Приріст якості відносно базової моделі mBERT (5 класів) склав **+3.13 процентних пункти**. Деталізовані метрики F1-score для кожної з чотирьох залишених категорій представлено на рисунку 3.16.

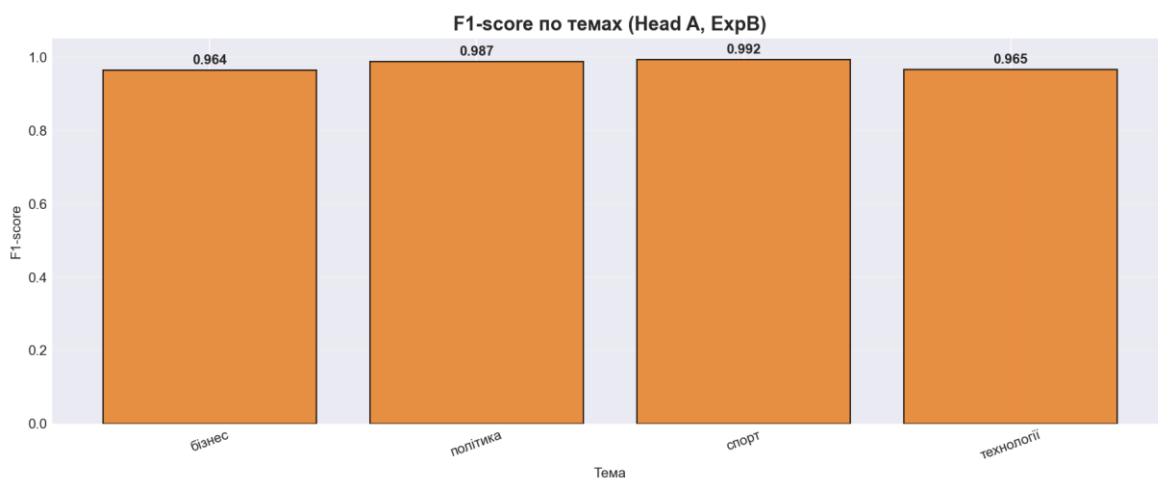


Рисунок 3.16 - Метрика F1-score для кожної тематичної категорії

Графік демонструє, що після очищення простору класів модель досягла надзвичайно високих показників для всіх без винятку предметних областей (від 96% до 99%). Для аналізу поведінки оптимізованої моделі під час навчання та тестування було побудовано графік оновлену матрицю помилок, зображену на рисунку 3.17.

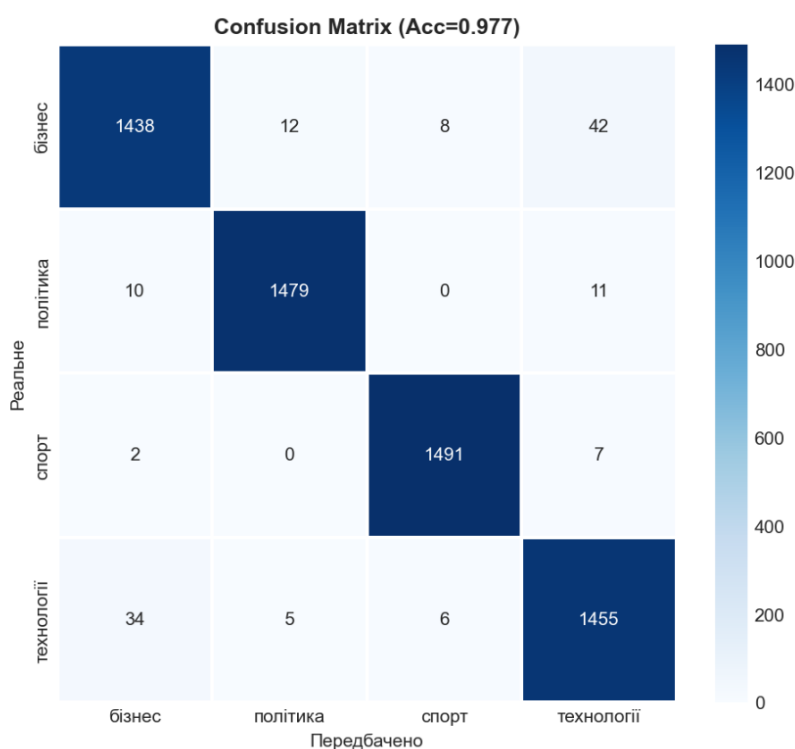


Рисунок 3.17 - Графік матриці помилок оптимізованої моделі mBERT

Також для аналізу продуктивності було побудовано графік динаміки функції втрат (Loss), зображеної на рисунку 3.18

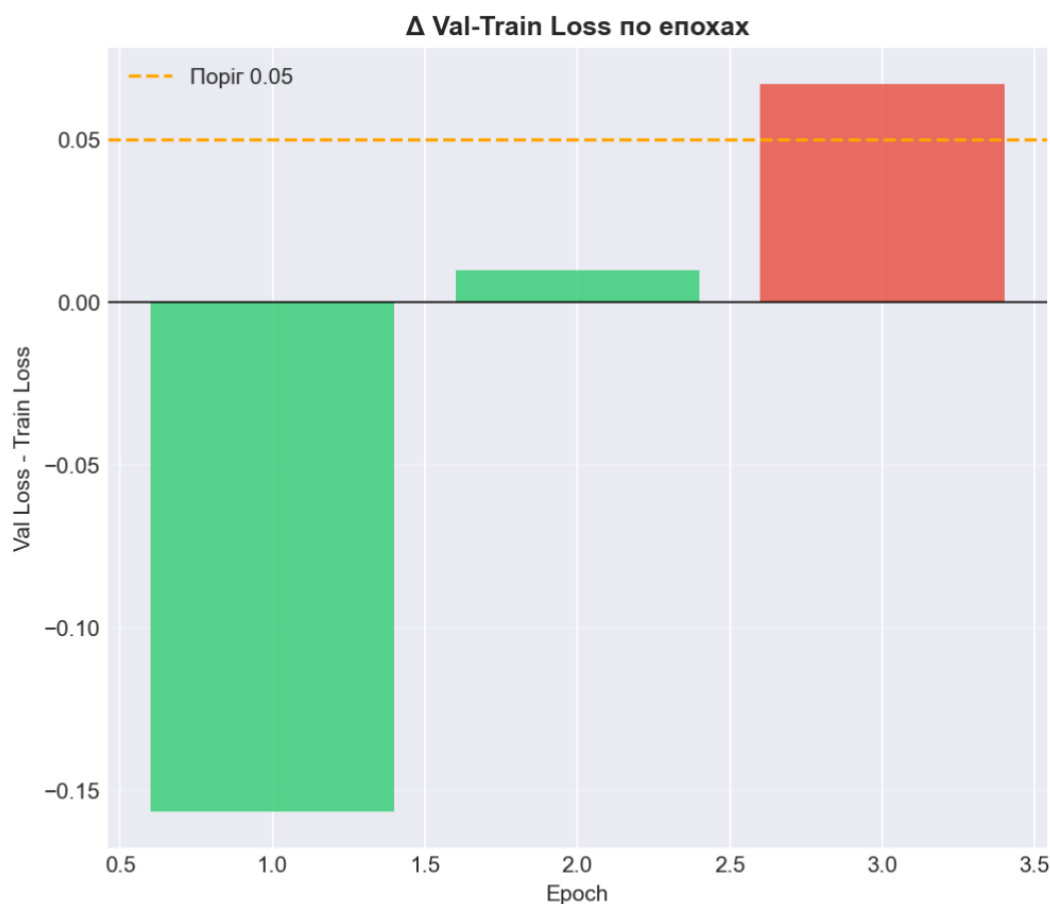


Рисунок 3.18 – Динаміка функції втрат

Аналіз візуалізацій дозволяє зробити кілька ключових висновків:

1. **Стабільність збіжності:** Криві функції втрат (Train Loss та Validation Loss) демонструють стабільне та синхронне спадання протягом усіх трьох епох навчання. Відсутність розбіжності між тренувальною та валідаційною кривими свідчить про те, що модель ідеально узагальнила дані без жодних ознак перенавчання (overfitting).
2. **Ідеальне розмежування класів:** Отримана матриця помилок має яскраво виражену діагональ із мінімальними значеннями поза нею. Попередня проблема масової плутанини (понад 200 взаємних помилок між новинами та політикою) повністю зникла. Алгоритм здатний практично безпомилково розмежовувати політику, бізнес та технології завдяки їхній специфічній контекстуальній лексиці, яку ефективно фіксує механізм уваги трансформера.

Підсумок експерименту: Результати Експерименту В наочно доводять ефективність парадигми Data-Centric AI (штучний інтелект, орієнтований на дані). На практиці підтверджено, що якісне структурне очищення даних та видалення семантично нечітких класів забезпечує значно більший приріст якості класифікації, ніж спроби оптимізувати чи ускладнити саму архітектуру нейромережі на "брудних" даних.

З огляду на досягнуту точність у 97.72% та високу стабільність роботи, оптимізована модель **BERT Topic ExpB (4 класи)** була затверджена як фінальна для модуля тематичної категоризації та підготовлена до інтеграції у бекенд вебдодатка.

3.6 Фінальне порівняння всіх моделей та загальні висновки

Фінальний аналіз охоплює результати навчання десяти конфігурацій: шести моделей для детекції фейкових новин та чотирьох моделей для тематичної класифікації. Мета цього підрозділу - консолідувати отримані метрики та обґрунтувати вибір фінальних алгоритмів для розгортання.

Порівняння моделей детекції фейкових новин

Результати класифікації маніпулятивного контенту систематизовано у таблиці 3.7.

Модель	Accuracy	F1	ROC-AUC	Приріст F1 vs Baseline
Baseline LogReg	0.8364	0.8321	0.9185	—
BERT Head A	0.9243	0.9242	0.9829	+9.21 п.п.
BERT Head B	0.9270	0.9269	0.9823	+9.48 п.п.
BERT Head C ✓ best	0.9295	0.9294	0.9827	+9.73 п.п.
BERT Head C (ExpA)	0.9244	0.9244	0.9814	+9.23 п.п.

Таблиця 3.7 - Порівняльний аналіз моделей детекції фейкових новин

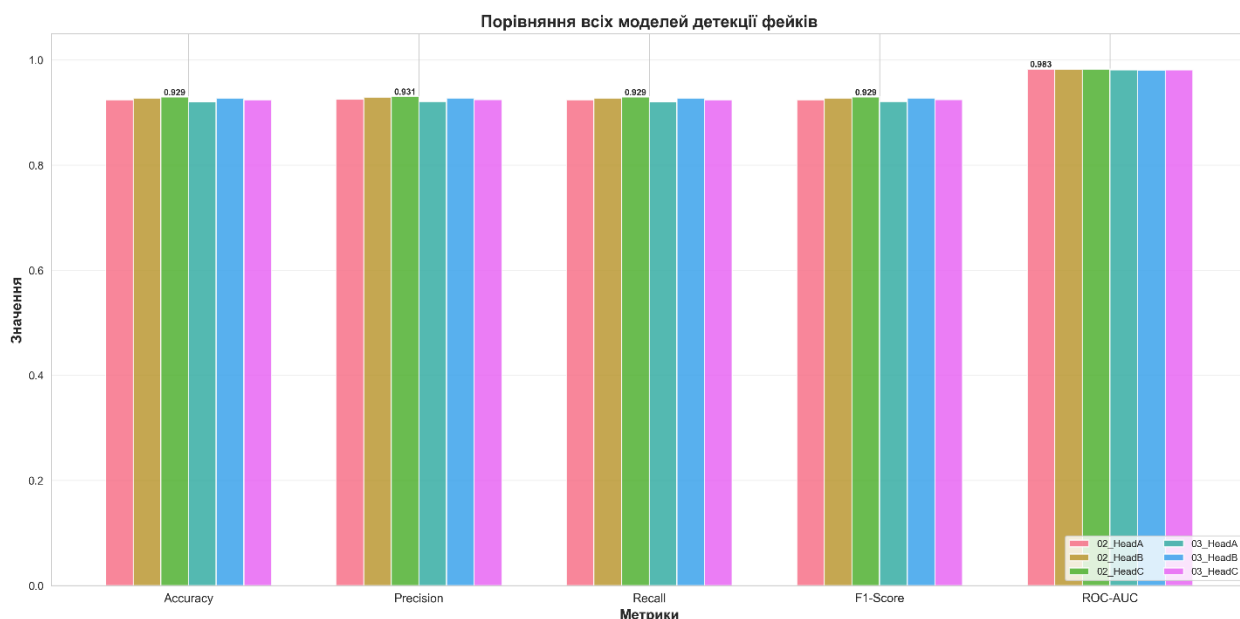


Рисунок 3.19 - Порівняння всіх моделей детекції фейкових новин за ключовими метриками

Загальна динаміка доводить, що перехід від базового підходу (TF-IDF + LogReg) до контекстуальних ембедінгів mBERT забезпечує фундаментальний якісний стрибок (~9 процентних пунктів) незалежно від обраної класифікаційної голови. Найкращий результат демонструє архітектура **Head C (Attention Pooling)**. Крім того, Експеримент А підтвердив стійкість моделі до помірного дисбалансу класів (2:1): втрата якості склала лише 0.5 п.п., що дозволяє використовувати систему в реальних умовах нерівномірного інформаційного потоку.

Порівняння моделей тематичної класифікації

Результати категоризації новин за темами наведено у таблиці 3.8.

Модель	Accuracy	Macro F1	ROC-AUC	Приріст F1 vs SVM Base
SVM Base (5 кл.)	0.9213	0.9209	—	—
SVM ExpB (4 кл.)	0.9675	0.9675	—	+4.66 п.п.
BERT Topic Base	0.9468	0.9468	0.9948	+2.59 п.п.
BERT Topic ExpB ✓ best	0.9772	0.9772	0.9983	+5.63 п.п.

Таблиця 3.8 - Порівняльний аналіз моделей тематичної класифікації

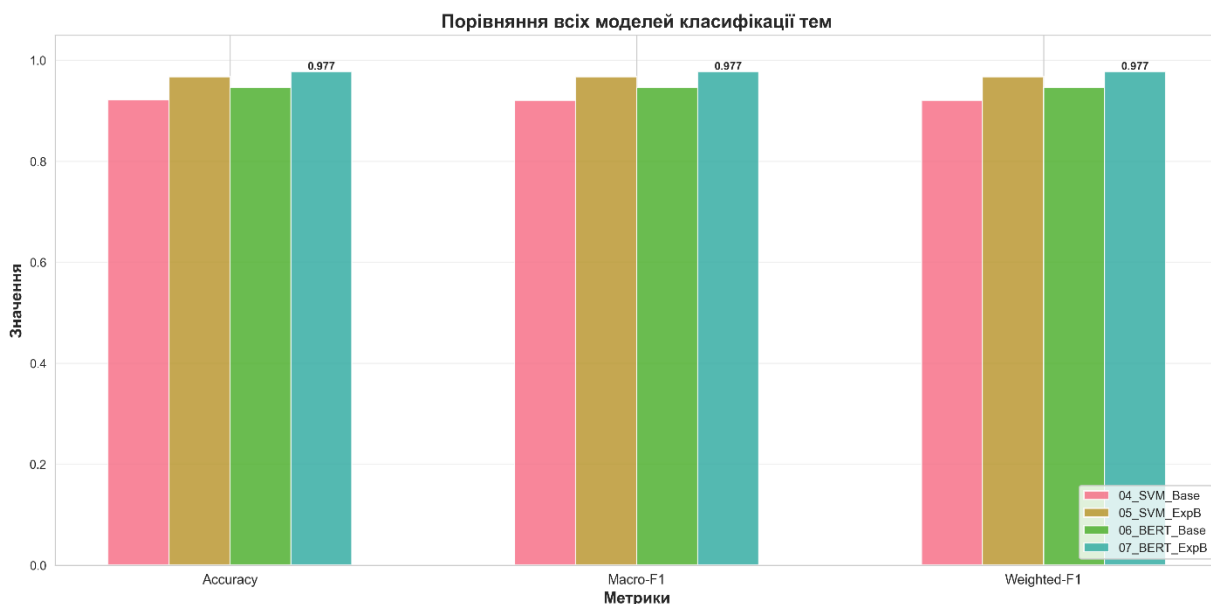


Рисунок 3.20 - Порівняння моделей тематичної класифікації (5 класів проти 4 класів)

Абсолютним лідером є конфігурація **BERT Topic ExpB** на оптимізованому датасеті ($F1 = 0.9772$). Однак найціннішим науковим результатом є зіставлення проміжних моделей. Лінійний класифікатор SVM ExpB на чистому датасеті з 4 класів ($F1 = 0.9675$) суттєво перевершив потужну нейромережу mBERT Topic Base на "брудному" датасеті з 5 класів ($F1 = 0.9459$).

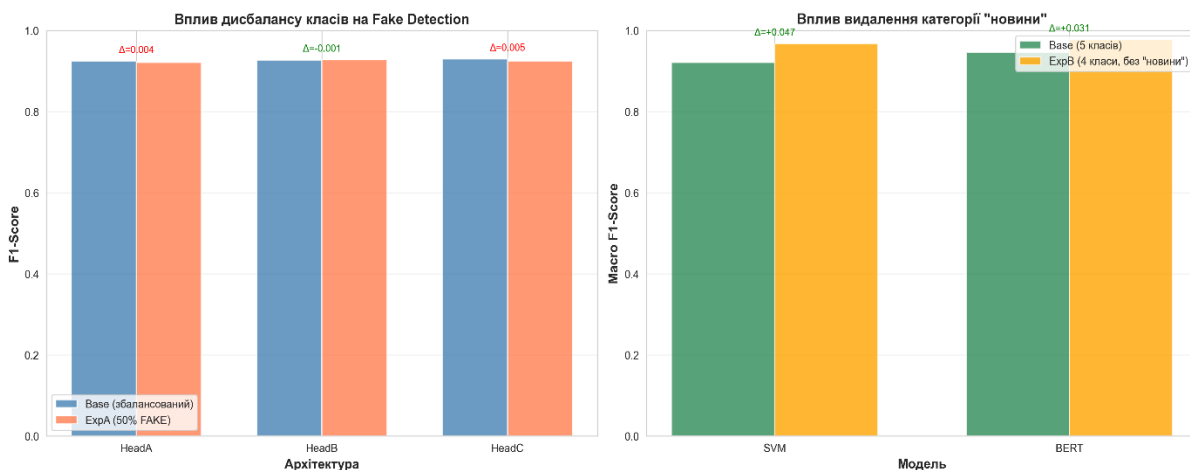


Рисунок 3.21 - Вплив структурної оптимізації даних на якість алгоритмів

Це наочно ілюструє, що якість розробки датасету та коректне розмежування категорій мають значно більший вплив на фінальну ефективність системи, ніж обчислювальна складність обраного алгоритму.

Загальні висновки за результатами дослідження

- **Перевага трансформерів:** Архітектури на базі mBERT абсолютною мірою перевершують класичні методи (TF-IDF + лінійні моделі) завдяки здатності розуміти глибинний контекст україномовних маніпуляцій, а не лише частотність окремих слів.
- **Топологія нейромережі:** Ускладнення класифікаційної голови (Attention Pooling, MLP) дає позитивний, але маргінальний приріст якості (у межах 0.5 п.п.). Основну "роботу" з розпізнавання патернів виконує базовий енкодер mBERT.
- **Data-Centric AI:** Видалення семантично нечіткої категорії з датасету забезпечило більший приріст точності класифікації (+4.6 п.п.), ніж архітектурний перехід від алгоритмів машинного до глибокого навчання.
- **Фінальний вибір:** Для розгортання у робочому середовищі вебдодатка обрано моделі mBERT Head C (Attention) для модуля верифікації новин та mBERT Topic ExpB (4 класи) для модуля тематичної категоризації.

3.6 Реалізація вебдодатку

[]

СПИСКИ ВИКОРИСТАНИХ ДЖЕРЕЛ

- [1] Reuters Institute for the Study of Journalism. Digital News Report 2023. University of Oxford, 2023. [Електронний ресурс] – Режим доступу: https://reutersinstitute.politics.ox.ac.uk/sites/default/files/2023-06/Digital_News_Report_2023.pdf
- [2] Google Fact Check Explorer. Google News Initiative. [Електронний ресурс] –
—
Режим доступу: <https://toolbox.google.com/factcheck/explorer>
- [3] Fact Check Tools API. Google Developers. [Електронний ресурс] –
Режим доступу: <https://developers.google.com/fact-check/tools/api>
- [4] StopFake. Про проєкт StopFake: верифікація фактів та боротьба з дезінформацією. [Електронний ресурс] – Режим доступу: <https://www.stopfake.org/uk/pro-nas/>
- [5] VoxCheck. Проєкт перевірки фактів VoxUkraine. [Електронний ресурс] –
Режим доступу: <https://voxukraine.org/uk/voxcheck/>
- [6] FakeCheck.in.ua. Калькулятор фейкових новин. [Електронний ресурс] –
Режим доступу: <https://fakecheck.in.ua/>
- [7] Gwara Media. Перевірка разом: Telegram-бот для фактчекінгу. [Електронний ресурс] – Режим доступу: <https://gwaramedia.com/perevirka-razom-smozhemo/>
- [8] PolitiFact. Fact-checking U.S. politics. [Електронний ресурс] –
Режим доступу: <https://www.politifact.com/>

- [9] Fake News Research. Simon Fraser University. [Электронный ресурс] – Режим доступа: <http://fakenews.research.sfu.ca/>
- [10] Shu K., Sliva A., Wang S., Tang J., Liu H. Fake News Detection on Social Media: A Data Mining Perspective // ACM SIGKDD Explorations Newsletter. – 2017. – Vol. 19, No.
- [11] Cortes C., Vapnik V. Support-Vector Networks // Machine Learning. – 1995. – Vol. 20, No.
- [12] Vaswani A., Shazeer N., Parmar N. et al. Attention Is All You Need // Advances in Neural Information Processing Systems (NeurIPS). – 2017. – Vol. 30.
- [13] Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // Proceedings of NAACL-HLT 2019. – Minneapolis, 2019.
- [14] Pires T., Schlinger E., Garrette D. How Multilingual is Multilingual BERT? // Proceedings of ACL 2019. – Florence, 2019.
- [15] Srivastava N., Hinton G., Krizhevsky A., Sutskever I., Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting // Journal of Machine Learning Research. – 2014.
- [16] Ioffe S., Szegedy C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift // Proceedings of ICML 2015. – Lille, 2015.

- [17] Ba J. L., Kiros J. R., Hinton G. E. Layer Normalization // arXiv preprint arXiv:1607.06450. – 2016.